Third year Methodological project - Advanced statistical engineering specialization

On an adaptive multi-fidelity Gaussian process metamodel using a collective learning strategy

Students :

Thomas GOUMONT
Léo GRUCHOCIAK
Inès BEN MOUSSA

Company supervisor:

Rémy PRIEM
AI expert for aeronautical systems,
Direction Générale de l'Armement

Academic supervisor :

Sébastien DA VEIGA
Professor-researcher, ENSAI

51 Rue Blaise Pascal, 35172 Bruz

September 2025 - January 2026

Lexicon

1. **Reliability analysis**
   A research field at the intersection of statistics and engineering, that studies the failure of physical systems, with a particular focus on estimating the probability that a system fails.

2. **Probability of failure**
   The probability that a system exceeds a critical threshold, leading to an unsafe or failed state.

3. **Limit state function**
   A mathematical function that separates the safe domain of a system from the failure domain. The boundary between these two regions is called the limit state.

4. **Surrogate model - Metamodel**
   A simplified and computationally efficient model used to approximate the behavior of a more complex and expensive model. In reliability analysis, it is commonly used to approximate the limit state function.

5. **Fidelity source**
   A model or source of information that provides an evaluation of the system response with a given level of accuracy and computational cost. High-fidelity models are accurate but expensive (in terms of computational resources, time, or financial expense), while low-fidelity models are cheaper and faster but less precise.

6. **Multi-fidelity metamodel**
   A metamodel that combines information from multiple models of different fidelity levels, to improve accuracy while reducing computational cost.

7. **Monte Carlo Simulation (MCS)**
   A numerical method based on random sampling that is used to estimate quantities of interest, such as the probability of failure.

8. **Design of Experiments (DoE)**
   An approach for selecting a set of input points at which a model or system is evaluated, with the goal of extracting as much information as possible using a limited number of experiments.

9. **Latin Hypercube Sampling (LHS)**
   A sampling technique that ensures a uniform coverage of the input parameter space.

10. **Gaussian process regression - Kriging**
    A statistical modeling technique used to build metamodels. It provides predictions of the system response together with an estimate of the associated uncertainty.

11. **Posterior Mean / Posterior Variance**
In Gaussian processes regression, the posterior mean represents the prediction of the metamodel, and the posterior variance quantifies the associated uncertainty.

12. **Active learning**
A modeling strategy where new data points are selected automatically in regions that are expected to be the most informative, in order to improve the model efficiently while minimizing the number of evaluations.

13. **Learning function**
A function that is used to select enrichment points for the metamodel.

14. **Collective learning function (CLF)**
A specific learning function that combines several criteria in order to select, in an optimal way, the sample points and the information source to use in a multi-fidelity metamodel.

Contents

1 Introduction **1**

2 An introduction to the reliability analysis framework **2**
2.1 Problem setup . . . 2
2.2 Monte Carlo Simulations . . . 2
2.3 The metamodeling approach . . . 2
2.4 Single-fidelity versus multi-fidelity models . . . 3

3 The Gaussian process metamodel **4**
3.1 Gaussian process regression . . . 4
3.2 The choice of a kernel function . . . 5
3.3 An illustration of the GP procedure . . . 5

4 Active learning methods in reliability analysis **6**
4.1 Active learning methods and learning functions . . . 6
4.1.1 The single-fidelity case and associated learning functions . . . 7
4.1.2 Extensions to the multi-fidelity setting . . . 8
4.2 The Design of Experiments . . . 8
4.3 Latin Hypercube Sampling . . . 9

5 The AMGPRA method **10**
5.1 A multi-fidelity model approach with a GP metamodel . . . 10
5.1.1 Defining the model's distributions . . . 10
5.1.2 The training point selection procedure . . . 11
5.2 Introducing the collective learning function . . . 12
5.3 The AMGPRA procedure . . . 13
5.4 Application: the AMGPRA method in practice . . . 14

6 Implementation and experimental framework **16**
6.1 General implementation framework . . . 16
6.2 mfEGRA implementation . . . 16
6.3 AMGPRA implementation . . . 19

7 Results and applications **21**
7.1 Reproducibility: testing our implementations on a reference example . . . 21
7.1.1 Numerical study based on Example 5.1 . . . 21
7.1.2 A modified example: shifted low-fidelity zero contours . . . 22

8 Case study: applying AMGPRA to the `solar` benchmark problem **25**
8.1 Physical and numerical context . . . 25
8.2 The `solar 10.1` instance . . . 26
8.2.1 Transformation into a reliability analysis problem . . . 27
8.2.2 Fidelity calibration and computational trade-off . . . 28
8.3 Numerical experiments and results on `solar 10.1` . . . 29
8.3.1 Experimental setup . . . 29
8.3.2 Fragility curve analysis: evolution of P_f with the overrun margin . . . 30
8.3.3 2D reliability mapping around the optimum . . . 31
8.3.4 Discussion . . . 33

9 Concluding remarks **34**

A Additional material on the AMGPRA method **35**
A.1 Closed form expression of the EFF learning function 35
A.2 Likelihood function optimization and hyperparameters estimation 35
A.3 On the implementation of the AK-MCS method 36

References **38**

Abstract

The assessment of system reliability is a critical issue in many engineering domains. Being able to anticipate whether a system will operate safely or fail based solely on the knowledge of experimental conditions can significantly reduce both development time and costs. In this context, reliability analysis commonly rely on surrogate models to approximate computationally expensive physical models.

Among these approaches, Gaussian processes surrogate models (or Kriging) play a central role due to their ability to provide not only predictions but also an estimate of the associated model uncertainty. This property is widely exploited in active learning strategies to iteratively select the most informative points for model refinement. Furthermore, the combined use of information sources with different levels of fidelity, referred to as high and low-fidelity sources, has been shown to further improve the efficiency of reliability analyses.

This report presents the Adaptive Multi-fidelity Gaussian Process for Reliability Analysis (AMGPRA) approach [1], a recently developed multi-fidelity Kriging-based method. AMGPRA relies on the simultaneous exploitation of surrogate models of different fidelities and introduces several innovations compared to the state of the art methods in multi-fidelity reliability analysis, which is represented here by the mfEGRA method [2]. Notably, AMGPRA employs a collective learning function to jointly select the next evaluation point and the corresponding fidelity source, leading to more accurate, or at least equivalent, estimates of the probability of failure, while reducing the overall computational cost.

Keywords: Reliability, Gaussian process, Metamodel, Fidelity models, Probability of failure, Learning functions, Adaptive Kriging, Active Learning.

Symbols and Notation

Symbol	Meaning
$\mathcal{X}$	Input space
$\mathcal{Y}$	Output space
$\mathbb{R}$	Set of real value numbers
$\mathbb{R}^d$	Set of d-dimensional real value vectors
$\mathbb{N}$	Set of positive integers
g_0	High-fidelity function
g_l	Low-fidelity function
$\mathcal{GP}$	Gaussian process
$k(\mathbf{x}, \mathbf{x}')$	Covariance kernel for $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$
$\hat{g}(0, \mathbf{x})$	High-fidelity GP model at $\mathbf{x} \in \mathcal{X}$
$\hat{g}(l, \mathbf{x})$	Low-fidelity GP model for the l-th fidelity source at $\mathbf{x} \in \mathcal{X}$
$\mu_0(\mathbf{x})$	Prior mean function of the high-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\mu_l(\mathbf{x})$	Prior mean function of the l-th low-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\mathbf{\Sigma}_0(\mathbf{x}, \mathbf{x}')$	Prior covariance kernel of the high-fidelity GP for $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$
$\mathbf{\Sigma}_l(\mathbf{x}, \mathbf{x}')$	Prior covariance kernel of the l-th low-fidelity GP for $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$
$G_P(0, \mathbf{x})$	Posterior distribution of the high-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$G_P(l, \mathbf{x})$	Posterior distribution of the l-th low fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\mu_P(0, \mathbf{x})$	Posterior mean of the high-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\mu_P(l, \mathbf{x})$	Posterior mean of the l-th low-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\sigma_P^2(0, \mathbf{x})$	Posterior variance of the high-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\sigma_P^2(l, \mathbf{x})$	Posterior variance of the l-th low-fidelity GP at $\mathbf{x} \in \mathcal{X}$
y^F	Hypothetical future simulated response at $\mathbf{x}^{n+1} \in \mathcal{X}$, with the information source l_F
$G_F\big(0, \mathbf{x}\big)$	Future distribution of the high-fidelity GP at $\mathbf{x} \in \mathcal{X}$, after adding a hypothetical new training point $(\mathbf{x}^{n+1}, l_F, y^F)$
$\mu_F(0, \mathbf{x})$	Posterior mean of the hypothetical future high-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\sigma_F(0, \mathbf{x})$	Posterior variance of the hypothetical future high-fidelity GP at $\mathbf{x} \in \mathcal{X}$
$\mathbf{S}$	Monte Carlo Population
$\Delta\mathbf{S}$	Additional number of candidate design samples to be added to S

1 Introduction

In many scientific and engineering domains, evaluating the behavior of a complex physical system under uncertainty is essential to ensure that it performs reliably throughout its operational life. For instance, consider a team of engineers designing a wind turbine. Beyond meeting physical and mechanical constraints during the design and manufacturing of its components, such as turbine blades, the assembled system must be assessed before deployment to identify the specific conditions under which the turbine performs correctly. Accordingly, experiments are conducted to evaluate how the system behaves. This process aims at assessing the **reliability** of the system, giving rise to a discipline at the intersection of physical engineering and statistics known as **reliability analysis**. One of the main objectives of this field is to evaluate the probability that the produced system will fail, leading to the notion of **probability of failure**. In reliability analysis, a large field of analysis is dedicated to the methods that can be used in order to estimate this probability [3].

The reliability analysis of a physical system can be performed thanks to experiments. However, running many experiments for a wind turbine is costly, and, therefore, obtaining a lot of information on the evolution of the physical system through sensors is not always possible. Scientists have found a way to resolve this issue, by using what are called **surrogate models** or **metamodels**, which refer to mathematical formulations of the physical system studied, in order to perform numerical analysis of the reliability of the system [4, 5]. Yet, obtaining information through real data thanks to sensors or through a numerical evaluation of a model will not give the same quality of analysis, real data obtained through sensors being often far more robust than the numerical simulations. Thus, a trade off needs to be made in order to obtain enough data in order to estimate the probability of failure of the system studied, while taking into account the costs of the information sources: this information will be costly with sensors or detailed simulation of the system, that will gave us robust data, or cheaper with simplified experiments or numerical methods, but with lower accuracy. The first ones are refered as **high-fidelity** sources, while the second as **low-fidelity** ones. A classical way in reliability analysis is to use only one high-fidelity source of information (*e.g.* sensors on the wind turbine blade), with an associated cost that is high, and others low-fidelity sources of information, which are cheaper and easy to evaluate. It is then common in reliability analysis to work with **multi-fidelity models** in order to estimate the probability of failure, referring to models that combine various sources of information, both low-fidelity and high-fidelity ones [6, 7].

The French *Direction générale de l'armement* (DGA) is a central actor in the development field of weapons and defense, contributing to the improvement of existing defense systems. They conduct research in reliability analysis to assess the behavior of defense systems, as system reliability is a key factor in evaluating overall performance. Therefore, DGA engineers are interested in recent methodologies for estimating the probability of failure of physical systems. One of the most recent method developed in the literature is the so-called **adaptive multi-fidelity Gaussian process for reliability analysis (AMGPRA)** method[1], proposed in 2022. The idea is to use a multi-fidelity model based on **Gaussian processes** in order to estimate the probability of failure of the physical system for which the simulation is costly. The method tries to select, in a single step, the optimal point to estimate and the type of fidelity to use while maximizing a global information gain, taking into account the costs of the various sources of information. The main innovation lies in the introduction of a new acquisition function, that enables the decision where and how to sample a costly model to improve the estimation of the stability of the system, called the collective learning function (CLF).

This report propose a detail explanation of the authors work, focusing on the methodology and highlighting the benefits of the AMGPRA method in the field of reliability analysis.

[1]See Zhang, C. Song, C., and Shafieezadeh, A., *Adaptive Reliability Analysis for Multi-fidelity Models using a Collective Learning Strategy*, in "Structural Safety", vol. 94, 2022.

2 An introduction to the reliability analysis framework

2.1 Problem setup

For a given physical system, the goal of reliability methods is to obtain a precise estimation of the probability of failure. Suppose that there is a random vector $\mathbf{X} \in \mathbb{R}^d$, with $d \in \mathbb{N}^*$, that describes the complex system studied, *i.e.* that this random vector describes the uncertainties of the system. In reliability analysis, a specific function

$$g : \mathbb{R}^d \to \mathbb{R},$$

is used, known as the *limit state function*, also called *performance function*. This function characterizes the safety margin of the system: its sign determines whether the system is in a safe state or in failure. More precisely, if $g(\mathbf{X}) \leq 0$, the system fails, and $g(\mathbf{X}) > 0$ otherwise. The points where $g(\mathbf{X}) = 0$ are referred as the *limit state boundary*. This function will be used to define the probability of failure of the system, which is given by :

$$P_f := \mathbb{P}(g(\mathbf{X}) \leq 0) = \int_{g(\mathbf{X})\leq 0} f(\mathbf{X}) d\mathbf{X}, \tag{1}$$

where $f(\mathbf{X})$ is the joint probability density function of $\mathbf{X}$. Reliability analysis methods try to build an estimate of this quantity.

In practice, the true limit state function g is generally unknown or inaccessible. Instead, one relies on a high-fidelity function g_0, which provides a costly but accurate approximation of g. In principle, the above integral could be estimated by repeatedly observing the real system under varying experimental conditions, which would correspond to evaluating the true limit state function g. However, such real-world observations are generally infeasible. Numerical simulations based on the high-fidelity function g_0, such as **Monte Carlo Simulation** (MCS) or **Importance Sampling** (IS) [8], are thus employed to approximate the probability of failure.

2.2 Monte Carlo Simulations

MCS [8] methods are used to obtain estimators of a certain quantity thanks to simulation of a large number of samples of the known random vector $\mathbf{X} \in \mathbb{R}^d$, denoting by $X_1, ..., X_{N^{MCS}}$ with $N^{MCS} \in \mathbb{N}^*$, and will evaluate at each sample if the system is failing, *i.e.* if $g(X_i) \leq 0$, $\forall\ i \in [|1, N^{MCS}|]$, and then take the empirical mean of the points in the Monte Carlo population that are actually failing. In our case, a MCS methods will estimate P_f as follows :

$$P_f = \int_{g(\mathbf{X})\leq 0} f(\mathbf{X})\, d\mathbf{X} \approx \frac{1}{N_{\text{MCS}}} \sum_{i=1}^{N_{\text{MCS}}} \mathbb{1}_{\{g_0(X_i)\leq 0\}} := \hat{P}_f, \tag{2}$$

where N^{MCS} denotes the number of Monte Carlo samples and $\mathbb{1}_{\{\cdot\}}$ is the indicator function.

This approach requires the probability distribution of the random vector $\mathbf{X}$ to be known or assumed in order to generate independent samples. Thanks to the law of large numbers, the Monte Carlo estimator $\hat{P}_f$ is unbiased and converges to the true probability of failure as N^{MCS} increases. However, achieving a sufficiently accurate estimate generally requires a large number of samples, leading to a high computational cost as each evaluation of the high-fidelity function g_0 is expensive. This limitation motivates the development of alternative strategies to estimate the probability of failure with a reduced number of high-fidelity evaluations.

2.3 The metamodeling approach

As discussed in the previous subsection, the practical application of standard Monte Carlo Simulation is often limited by the high computational cost associated with repeated evaluations of the high-fidelity

function g_0. To alleviate this issue, an alternative strategy consists in replacing the expensive g_0 function by a cheaper approximation $\hat{g}_0$. This idea motivates the use of metamodel-based approaches in reliability analysis [9].

> **Definition 1**: A **metamodel**, also referred to as a **surrogate model**, is a simplified or approximate model that aims to replicate the behavior of a more complex model while being much faster to compute. In reliability analysis, metamodels are widely used to approximate the limit state function and enable efficient estimation of the probability of failure.

In practice, the metamodel is constructed from a limited number of evaluations of the high-fidelity limit state function g_0, and is intended to approximate its response over the input space. Once built, the surrogate model $\hat{g}_0$ can be substituted for the original function in the estimation procedure, allowing Monte Carlo simulations to be performed efficiently. However, in many engineering and scientific applications, the high-fidelity model is not the only source of information available. It is often accompanied by one or several low-fidelity models, which provide coarser but significantly cheaper approximations of the same physical system. Although low-fidelity models are often not accurate enough to be used directly for reliability assessment, they provide useful qualitative information at a negligible computational cost that allows us to run them a large number of times.

This observation has motivated the development of multi-fidelity metamodeling approaches [6, 7], whose objective is to combine information from different levels of fidelity. By exploiting both the accuracy of a small number of high-fidelity evaluations and the abundance of low-fidelity data, these approaches aim to build an accurate approximation $\hat{g}_0$ of the high-fidelity response while keeping the overall computational effort under control. The resulting multi-fidelity surrogate can then be used in place of the original high-fidelity model to perform Monte Carlo simulations efficiently and estimate the probability of failure.

Various types of metamodels exist in reliability analysis, relying on either a single information source or multiple fidelity levels. Among them, the Gaussian process metamodel, or *Kriging*, is particularly popular due to its appealing theoretical properties and its ability to quantify prediction uncertainty [10]. This probabilistic formulation extends naturally to the multi-fidelity setting. **Section 3** provides a detailed presentation of Gaussian process regression method, that is used behind every Kriging-based metamodels.

2.4 Single-fidelity versus multi-fidelity models

In a single-fidelity framework, the surrogate model is constructed exclusively from evaluations of the high-fidelity limit state function g_0. While this guarantees high accuracy, it still requires a significant number of expensive evaluations, even when compared to standard Monte Carlo Simulation [11]. In reliability analysis, the failure domain is typically very small and hard to locate *a priori* [7], so a single-fidelity approach often fails to adequately explore the input $\mathcal{X}$ space under a limited computational budget. As a result, the surrogate can be poorly informed in the regions of interest (*i.e.*, near the limit state boundary), leading to inaccurate estimates of the failure probability.

In contrast, multi-fidelity models explicitly exploit the availability of cheaper low-fidelity simulations $(g_l)_{l=1}^K$ to enhance the construction of the surrogate. Although these low-fidelity models are less accurate, they provide valuable structural information over the entire input domain and can be evaluated in large quantities. By combining a small number of expensive high-fidelity evaluations with abundant low-fidelity information, a multi-fidelity metamodel can achieve a level of accuracy that single-fidelity methods cannot reach under the same computational budget [9]. This combination improves both the global representation of the system and the local refinement around the failure region, making multi-fidelity surrogates

particularly suitable for reliability estimation.

This is precisely the motivation behind advanced algorithms such as AMGPRA [1], which explicitly integrate multiple sources of information to construct an accurate surrogate while minimizing the use of costly high-fidelity evaluations.

3 The Gaussian process metamodel

In the literature of metamodel–based approaches, one[2] have proposed Gaussian processes regression as an efficient framework for jointly modeling low-fidelity and high-fidelity data. In particular, one of the main advantages of Gaussian process regression is its ability to provide a quantification of predictive uncertainty through the posterior variance [10]. In the following section, we try to give an overview of Gaussian process regression, based on the Bayesian formalism [12].

3.1 Gaussian process regression

Let's consider the general regression framework, meaning that the output space is $\mathcal{Y} = \mathbb{R}$, and where there is an input domain $\mathcal{X} = \mathbb{R}^d$, a given sample of size $n \in \mathbb{N}^*$ denoted by $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$ that represents the observations, supposed independent and identically distributed (i.i.d), and suppose that there is an unknown (or *blackbox*) function $f : \mathcal{X} \to \mathbb{R}$, with the following relationship : $\forall\ i \in [|1, n|]$, $y_i = f(x_i) + \epsilon_i$, with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. Our goal is to predict $f(\mathbf{x}^*)$ for some new values $\mathbf{x}^* \in \mathcal{X}$ given.

Gaussian process regression (denoted GP in what follows), also known as Kriging[3] in engineering literature, is a nonparametric approach to regression, based on the Bayesian formalism. In classical Bayesian regression, a *prior distribution* is placed in the model parameters, *i.e.* a distribution describing their behavior before observing any data. Bayes' theorem is then used to obtain the *posterior distribution* of the parameters given the observations.

In GP regression, instead of placing a prior on a finite set of parameters, we place a prior directly over the space of functions [10]. In other words, we assume that the unknown function f is a sample from a Gaussian process prior. Conditioning this prior on the observed data then yields the *posterior distribution* of the function, from which predictions can be made. Before deriving this posterior, let's recall the definition of a Gaussian process.

> **Definition 2**: A **Gaussian process** is a stochastic process, *i.e.* a collection of random variables $\{f(x)\}_{x\in\mathcal{X}}$ (potentially infinite), such that the vector $(f(x_1), \dots, f(x_n))$ follows a multivariate Gaussian distribution for any finite set of inputs $x_1, \dots, x_n$.

A Gaussian process is fully specified by its mean function and its covariance function, also called *covariance kernel*: $\forall\ \mathbf{x}, \mathbf{x}' \in \mathcal{X}$,

$$\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})] \tag{3}$$

$$k(\mathbf{x}, \mathbf{x}') = \mathrm{Cov}(f(\mathbf{x}), f(\mathbf{x}')) = \mathbb{E}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{x}') - \mu(\mathbf{x}'))] \tag{4}$$

The Gaussian process then writes: $f \sim \mathcal{GP}(\mu_{pr}(\mathbf{x}), k_{pr}(\mathbf{x}, \mathbf{x}'))$. In a GP model, the prior distribution simply implies specifying a prior mean function and a covariance kernel. The posterior distribution is then obtained by conditioning the GP prior of f on the observations in $\mathcal{D}_n$, leading to a random variable that follows a Gaussian distribution in each point (also called a *Gaussian random field*), *i.e.* for a new input $\mathbf{x}^* \in \mathcal{X}$:

[2] See for example [13]

[3] In the name of D.G. Krige, who introduced the method in *A statistical approach to some basic mine valuation problems on the Witwatersrand*, "Journal of the Southern African Institute of Mining and Metallurgy", vol.52, 1951.

$$f(\mathbf{x}^*)|\mathcal{D}_n \sim \mathcal{N}(\hat{\mu}_{GP}(\mathbf{x}^*), \hat{\sigma}^2_{GP}(\mathbf{x}^*)),$$

where $\hat{\mu}_{GP}$ and $\hat{\sigma}^2_{GP}$ are the posterior mean and posterior covariance functions used for prediction. These two quantities are obtained thanks to formulas that can be derived from the posterior distribution:

$$\begin{cases} \hat{\mu}_{GP}(\mathbf{x}^*) = \mu(\mathbf{x}^*) + k_n(\mathbf{x}^*)^T(\Sigma_n + \sigma^2 I_n)^{-1}(Y_n - \mu(X_n)) \\ \hat{\sigma}^2_{GP}(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - k_n(\mathbf{x}^*)^T(\Sigma_n + \sigma^2 I_n)^{-1}k_n(\mathbf{x}^*) \end{cases} \tag{5}$$

where $X_n = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ and $Y_n = (y_1, \dots, y_n)$ are respectively the vector of observed variables and the vectors of observed outputs, $\Sigma_n = [k(x_i, x_j)]_{i=1}^n \in \mathbb{R}^{n\times n}$ the prior covariance matrix evaluated at the observed data points, and $k_n(\mathbf{x}^*) = (k(\mathbf{x}^*, x_1), \dots, k(\mathbf{x}^*, x_n))^T$ the vector of covariance between the predicted point and the observed ones.

Looking at Equation 5, we see that the posterior mean function is a linear combination of the observations y_i from $\mathcal{D}_n$. The posterior variance is a representation of the uncertainty at the predicted point: it is low near observed points and large far from the data.

3.2 The choice of a kernel function

GP model predictions are quite dependent on the choice of the covariance kernel in the prior GP. This kernel enables the similarity between two points $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$ to be compared. It should be chosen wisely, and there is a large choice of possibilities for the kernel, allowing a wide range of covariance functions to be built. One of the most used in applications (but not necessary the optimal one) is the Radial Basis Function (RBF) kernel, or Gaussian kernel, which is defined in dimension one by:

$$k(\mathbf{x}, \mathbf{x}') = {\sigma_f}^2 \exp\left(-\frac{(\mathbf{x} - \mathbf{x}')^2}{2\,h^2}\right), \tag{6}$$

with ${\sigma_f}^2$ the variance of the Gaussian process f[4], and $h > 0$ an hyperparameter (usually called the kernel *width*) that represents the correlation length (the distance from which the two points stay correlated), controlling the smoothness of the GP. Usually, it is estimated either by cross-validation or by maximizing the marginal likelihood of the GP model, *i.e.* using *Maximum Likelihood Estimation* (MLE) [14]. This kernel is said to be *isotropic*, meaning that there is only one hyperparameter common to all features. It highlights the necessity to normalize the features beforehand, and limit the prediction accuracy in practice, *e.g.* if there exists features that have a higher predictive power than others.

In order to be a covariance kernel, a mapping $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ must satisfy some conditions.

> <u>**Definition 3**</u>: A mapping $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is a **covariance kernel** if and only if it satisfies the two following properties:
> (i) (**Symmetry**) $\forall\, \mathbf{x}, \mathbf{x}' \in \mathcal{X}, k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x})$
> (ii) (**Positivity**) $\forall\, n \in \mathbb{N}^*, \forall\, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)} \in \mathcal{X}, \forall\, a_1, \dots, a_n \in \mathbb{R}, \sum_{i=1}^n \sum_{j=1}^n a_i a_j k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \geq 0$

Very often, the covariance kernel is chosen *a priori*, or by a cross-validation procedure among a certain number of possible kernels.

3.3 An illustration of the GP procedure

To illustrate how the GP regression works in practice, we consider a simple one-dimensional function $f(x) = \sin(3x) + 0.5\cos(5x)$, and a set of input points, called the Design of Experiments[5], composed of only

[4] The signal variance ${\sigma_f}^2$ is typically estimated jointly with the other kernel hyperparameters using either MLE or *Maximum a Posteriori (MAP)* methods.

[5] See the **Sub-section 4.2** for a detailed explanation.

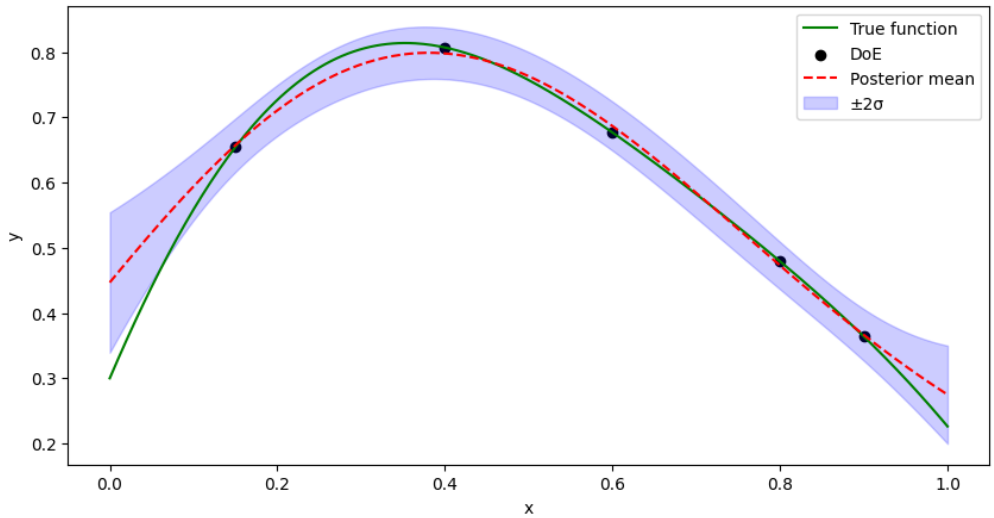

Figure 1: Illustration of a GP regression on the function $f(x)$, with a DoE of 5 points

5 evaluations of this function. The GP surrogate is then trained on these observations, using a Gaussian kernel and hyperparameters estimated by MLE. The model is supposed to be noisy. Figure 1[6] shows in green the true function, the sampled DoE points (the black dots), and the GP posterior mean (the red dashed line) together with a confidence band corresponding to $\hat{\mu}_{GP}(\mathbf{x}^*) \pm 2.\hat{\sigma}^2_{GP}(\mathbf{x}^*)$ (the blue shaded area).

This example highlights the key properties of GP regression. Firstly, the posterior mean passes really close to the observed data points, because the level of noise in the model is really small. The quantification of uncertainty is obtained thanks to the posterior variance: it is small in the vicinity of observed points, and increases in regions where no data is available. This reflects the uncertainty in the model. On the other hand, the smoothness of the GP prediction directly results from the smoothness of the chosen covariance kernel (here, the RBF kernel). A different kernel would induce a different structure of correlation between points. Finally, we see that, as new data points are added, both the posterior mean and posterior variance are updated analytically (thanks to the Bayesian formulation of the problem), which makes GP models particularly attractive for sequential sampling or active learning.

This simple example illustrates the exact mechanism used later in the AMGPRA algorithm: at each iteration, the GP surrogate provides both a prediction of the limit state function and an associated uncertainty, which are jointly exploited to decide where the next evaluation should be performed.

4 Active learning methods in reliability analysis

The AMGPRA method was developed in a context where applications of multi-fidelity metamodels are scarce. However, some authors have published interesting techniques that need to be highlighted, as they were used as foundations for the AMGPRA method. The goal of this section is to introduce smoothly all the final requirements needed to understand how the AMGPRA methodology works. In particular, AMGPRA belongs to the class of active learning methods, that needs to be discussed.

4.1 Active learning methods and learning functions

In surrogate-based reliability analysis, the main difficulty lies in determining where to evaluate the expensive limit state function g_0 in order to improve the accuracy of the metamodel, specifically near the failure boundary. This led to the development of *active learning methods* [15]. These methods consist

[6]This figure was produced with the Python library *gaussian_processes*, and the function GaussianProcessRegressor.

of sequential algorithms that iteratively select new evaluation points based on the current state of the metamodel [16]. Instead of sampling the input space blindly, active learning guides the exploration of $\mathcal{X}$ toward the most informative regions, typically those where the uncertainty of the metamodel is high. The user does not choose the training sample where data points will be evaluated, but the metamodel does it itself. For multi-fidelity active learning methods, in addition to the next training point, the algorithm also needs to select the information source where the point will be evaluated, weither high or low-fidelity.

To do so, active learning methods rely on **learning functions**, also called **acquisition functions**, which aim to quantify the information that is contained in each candidate point of the input space.

> **Definition 4**: A **learning function** is a mapping $\eta : \mathcal{X} \to \mathbb{R}$ that assigns a real valued number to points of the domain $\mathcal{X}$. This assigned score is used to select the points of the input space at which the model will be evaluated.

A learning function assigns a quantitative value, seen as a score, to every point $\mathbf{x}$ of the domain, and the next point where the metamodel will be evaluated is chosen as the maximizer of this score, *i.e.* they are used in order to decide where to go in $\mathcal{X}$ in order to enrich the metamodel. In the case of GP metamodel approaches, learning functions based on the posterior variance and on the closeness of the point to the estimated limit state boundary have been developed [17]. The regions where $\hat{\sigma}^2_{GP}(\mathbf{x})$ is large and $\hat{\mu}^2_{GP}(\mathbf{x})$ is close to 0 are indeed candidates for refinement, leading to efficient identification of the limit state boundary while minimizing the number of expensive evaluations of g_0.

4.1.1 The single-fidelity case and associated learning functions

Before AMGPRA, other reliability analysis methods based on GP surrogate and using an active learning methodology have been developed, using specific learning functions.

First, in the single high-fidelity case, the *Efficient Global Reliability Analysis* (EGRA) method was proposed by B.J. Bichon et al.[18]. Their idea was to directly target the input regions where the GP uncertainty affects the failure probability estimate, improving the efficiency of single-fidelity reliability computations. They introduce a particular learning function, the *Expected Learning Function (EFF)*, based on the expectation of the feasibility of a point, given for an untried point $\mathbf{x} \in \mathcal{X}$ by[7]:

$$EFF(\mathbf{x}) = \int_{a-\epsilon(\mathbf{x})}^{a+\epsilon(\mathbf{x})} (\epsilon(\mathbf{x}) - |a - y|).\phi(y\,; \hat{\mu}_{GP}(\mathbf{x}), \hat{\sigma}^2_{GP}(\mathbf{x}))\mathrm{d}y, \tag{7}$$

where ϕ is the standard normal probability density function, $\hat{\mu}_{GP}(\mathbf{x})$ and $\hat{\sigma}^2_{GP}(\mathbf{x})$ are respectively the posterior mean and posterior variance of the input point $\mathbf{x}$, $a = 0$, and $\epsilon(\mathbf{x}) = 2\sigma^2_{GP}(\mathbf{x})$. The term $(\epsilon(\mathbf{x}) - |a - y|)$ is used to measure the proximity between the point $\mathbf{x}$ and the limit state boundary $g(\mathbf{x}) = a = 0$, providing an appropriate way to select the next point to evaluate.

Another approach was proposed by Echard et al.[17], in their *AK-MCS* method (for *Active learning Kriging - Monte Carlo Simulation*). It is still a single-fidelity method combining GP regression, but with another learning function to refine the surrogate near the failure boundary, the *U learning function*, which is defined as:

$$U(\mathbf{x}) = \frac{|\hat{\mu}_{GP}(\mathbf{x})|}{\hat{\sigma}_{GP}(\mathbf{x})} \tag{8}$$

where $\hat{\sigma}_{GP}(\mathbf{x})$ is the posterior standard error at the untried point $\mathbf{x}$. This function is interesting as it gives a more important weight to the input points that are close to the predicted limit state. This function is used in AK-MCS as a *refinement criterion*: a small value of $U(\mathbf{x})$ indicates that the sample point is

[7]Note that a closed form for the expression in Equation (7) can be derived, and can be found is in the Appendix section. Obtaining a closed form for the learning function is a huge advantage for computations, as many learning functions are only approximated by integrals, leading to higher computational cost.

simultaneously close to the predicted limit state surface and subject to large epistemic uncertainty. A small value of $U(\mathbf{x})$ implies that either $|\hat{\mu}_{GP}(\mathbf{x})|$ is close to 0, *i.e.* $\mathbf{x}$ is close to the limit state boundary, and/or that $\hat{\sigma}_{GP}(\mathbf{x})$ is large, meaning that the evaluated point is subject to great uncertainty. Therefore, in that case, we look for the minimum of the learning function, not for the point that maximizes it. According to their method, the next point to be evaluated by the high-fidelity model g_0 is chosen by solving the problem:

$$\mathbf{x}^\star = \arg\min_{\mathbf{x}\in\mathcal{X}} U(\mathbf{x}). \tag{9}$$

This selected point is exactly the one where the risk of failure is the highest, and that is why it needs to be evaluated.

4.1.2 Extensions to the multi-fidelity setting

As explained in **section 2.4**, the single-fidelity setting is limited by computational complexity, leading to a poor exploration of the input space. To overcome this problem, multi-fidelity methods have been proposed [6, 9, 19], extending the methods explained in **section 4.1**.

In particular, a multi-fidelity GP metamodel has been proposed by Poloczek et al.[20], that combines information from multiple sources into one GP metamodel. It assumes that $K+1$ models of the limit state function are available, with $K \in \mathbb{N}^*$ low-fidelity models $(g_l)_{l=1}^K$, and only one high-fidelity model g_0. The GP metamodel is then able to combine both sources of information in order to provide a posterior mean and a posterior variance at any location $\mathbf{x}$.

The AMGPRA method was built on this development. The full explanation of the meta-modeling scheme will be presented in **section 5**. However, AMGPRA is not the only method that tries to extend the GP metamodel to the multi-fidelity framework. Chaudhuri et al.[2] proposed *mfEGRA*, a direct extension of the EGRA method into multi-fidelity models, considered as state of the art of the multi-fidelity case. The idea is then to select both the training point and the information source, high or low-fidelity. The proposed method relies on the EFF function (Equation (7)), and determines the next training point by solving an optimization problem. In order to find the information source to be used, mfEGRA used the *Kulback-Leibler divergence* between the actual GP metamodel and a future one where an information source is used to simulate the next training point to be evaluated.

Despite its innovative extension, mfEGRA suffers from some issues. First, the optimization problem for finding the next training point does not always select the most informative next evaluation point, (*i.e.* the one that contains the largest quantity of information about the limit state, or the one with the highest uncertainty), and is seen as computationally expensive. Moreover, the main problem of the mfEGRA method is that the selection of the information source and the next training point are seen as two independent procedures, which can lead to a sub-optimal selection of the next training point.

Trying to overcome these issues, the AMGPRA method has been proposed. Its main contribution is the development of an algorithm that is capable of selecting at the same time the next training point to be evaluated along with the source of information to be used, thanks to a new learning function that chooses these two elements simultaneously.

Before diving into the full explanation of the proposed algorithm, some important vocabulary regarding the training point selection scheme is explained in the rest of this part.

4.2 The Design of Experiments

In order to perform the reliability analysis using a surrogate model, we need an initial set of data points on which the physical model will be evaluated. This initial set is called the **Design of Experiments (DoE)**:

Definition 5: In reliability analysis, the **Design of Experiments** is a collection of input points

$$\{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathcal{X}$$

at which the high-fidelity model g_0 and the low-fidelity models g_l are evaluated in order to construct a surrogate model.

This set of points is used to initialize the surrogate model. In the AMGPRA method, the DoE points are used to initialize the GP metamodel, providing the initial data used to estimate the hyperparameters governing the correlation structure, but also to stabilize the numerical estimation of those hyperparameters of the GP, since the estimation requires a minimum amount of points[8].

Once the initial DoE is fixed, AMGPRA becomes **adaptive**, *i.e.* that new evaluation points are added iteratively to the DoE, guided by the predicted uncertainty from the GP posterior variance. The method is adaptive because the GP metamodel is updated at each iteration, and the next point to be added to the DoE is taken while considering the actual uncertainty and its position from $g_0(\mathbf{x}) = 0$, meaning that the algorithm adapts itself to what it learns from the input space. This characteristic allows the algorithm to focus on model evaluations in the most informative regions of the input space for estimating the probability of failure P_f.

To generate the initial DoE, a random sampling strategy in the input space is applied, assuming a uniform distribution over $[0, 1]^d$, after normalizing[9] the input variables. However, simple random sampling may leave large regions of $\mathcal{X}$ unexplored, and the procedure may miss informative points to estimate the probability of failure. That is why a stratification procedure is applied: unlike simple sampling method, stratification is a sampling scheme that divides the input space into intervals, and one precise point is sampled in each interval. This enables better space-filling properties and reduces the risk of clustering of some points in the DoE. In the AMGPRA method, this stratification scheme is performed thanks to the Latin Hypercube Sampling (LHS) method.

4.3 Latin Hypercube Sampling

Latin Hypercube Sampling (LHS)[21] is a stratified sampling technique commonly used to generate an initial space-filling DoE for high-dimensional domains, *i.e.* when d is especially large. The idea is to try to explore as much as possible the input space during the sampling procedure. To do so, the random sampling is done uniformly in each dimension of the inputs, allowing complete exploration of $\mathcal{X}$. Unlike simple random sampling, which may leave large regions of the input space unexplored, LHS ensures stratification in every dimension. For example, to generate n points in a d-dimensional input space, each input dimension d is divided into n equally sized intervals. One value is sampled uniformly at random inside each interval, and the sampled values from different dimensions are then combined through a random permutation, to form the final n final points.

This technique guarantees that every marginal distribution is uniformly covered, while the (unknown) joint distribution achieves good overall dispersion. The resulting n-points DoE is then evenly spread across the entire input domain, which is very important to obtain an accurate surrogate model.

In the AMGPRA method, LHS is used for two different roles. Firstly, it is used to generate the initial DoE, where the high-fidelity and the low-fidelity models are evaluated to build the first GP metamodel. Furthermore, LHS is also used in order to generate a large set of candidate points, on which a specific function, the *collective learning function*, is evaluated. Those points are never used to evaluate the high or

[8] For example, in order to perform the MLE of the hyperparameters of the covariance kernel.

[9] An affine transformation is applied before constructing the DoE such that each input variables take its values in $[0, 1]$, in order to avoid dimensional issues.

low-fidelity models, but only to identify which point seems to be the most informative on the limit state function, in order to add it to the initial DoE.

The AMGPRA algorithm stops adding new points to the DoE once the GP metamodel is sufficiently accurate, meaning that the variance near the limit state boundary $g_0(\mathbf{X}) = 0$ is sufficiently small, and the learning function that is used no longer detects interesting points to be added to the DoE. It is here that a MCS is finally performed on the GP metamodel, to obtain an estimate of P_f. This final MCS is cheap because it is run on the GP surrogate.

5 The AMGPRA method

This section is devoted to the detailed explanation of the studied method. The first part explains the adopted GP metamodel approach, before giving details on the full selection procedure. At the end, an application of the method is analyzed on a simple multi-fidelity case.

5.1 A multi-fidelity model approach with a GP metamodel

As explained in the previous section, the proposed AMGPRA method falls into the field of active learning multi-fidelity approaches, with a GP metamodel. Authors justified its development in order to try to obtain better results than the state of the art mfEGRA method, while trying to solve the associated issues. To understand how this is done, we need to formalize the proposed meta-modeling approach.

5.1.1 Defining the model's distributions

Suppose that we have $K + 1$ different models of the limit state function, with the hight-fidelity model corresponding to $l = 0$, the rest corresponding to the low-fidelity models. Let $\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$ denote the input random vector. Firstly, we start by defining the high-fidelity model by:

$$\hat{g}(0, \mathbf{x}) \sim \mathcal{GP}(\mu_0(\mathbf{x}), \Sigma_0(\mathbf{x}, \mathbf{x}'))$$

where $\mu_0(\mathbf{x})$ and $\Sigma_0(\mathbf{x}, \mathbf{x}')$ denote respectively the prior mean function and the prior covariance kernel of the high-fidelity Gaussian process. In what follows, we will denote by $\hat{g}(l, \mathbf{x})$ the GP surrogate of the $l = 1, ..., K$ low-fidelity models $g(l, \mathbf{x})$. Subsequently, each low-fidelity model is not modeled directly, but through its discrepancy with respect to the high-fidelity model. For $l = 1, \ldots, K$, the discrepancy is defined as

$$\delta_l(\mathbf{x}) = \hat{g}(l, \mathbf{x}) - \hat{g}(0, \mathbf{x}) \sim \mathcal{GP}(\mu_l(\mathbf{x}), \Sigma_l(\mathbf{x}, \mathbf{x}')) \,,$$

where $\mu_l(\mathbf{x})$ and $\Sigma_l(\mathbf{x}, \mathbf{x}')$ denote respectively the prior mean function and the prior covariance kernel of the discrepancy Gaussian process.

Here, $\delta_l(\mathbf{x})$ is supposed independent of the high-fidelity model and other low-fidelity models. Moreover, $\Sigma_l(\mathbf{x}, \mathbf{x}')$ describes how the bias varies across the input space, and $\mu_l(\mathbf{x})$ represents the general trend of the modeling error. It is important to note that in this study $\mu_l(\mathbf{x})$ is set to 0 to reflect the absence of belief for the bias direction regarding the high-fidelity model. However, alternative choices are possible, and $\mu_l(\mathbf{x})$ may be estimated, for instance, using MLE.

As the high-fidelity Gaussian process $\hat{g}(0, \mathbf{x})$ and the discrepancy process $\delta_l(\mathbf{x})$ are assumed to be independent, their sum

$$\hat{g}(l, \mathbf{x}) = \hat{g}(0, \mathbf{x}) + \delta_l(\mathbf{x})$$

is also a Gaussian process. Consequently, for any low-fidelity level $l = 1, \ldots, K$ and for any finite collection of input points $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \in \mathbb{R}^d$, the random vector

$$\Big(\hat{g}(l, \mathbf{x}_1), \ldots, \hat{g}(l, \mathbf{x}_n)\Big)$$

follows a multivariate normal distribution.

By setting $\delta_0(\mathbf{x}) = 0$, the multi-fidelity model can be expressed as a single Gaussian process defined over the augmented input space $\mathcal{Z} = \{0, \dots, K\} \times \mathbb{R}^d$, where we specify the following priors, corresponding to what has been said in Equation (3):

$$\hat{g}(l, \mathbf{x}) \sim \mathcal{GP}\Big(\mu_{\text{pr}}(l, \mathbf{x}), \Sigma_{\text{pr}}\Big((l, \mathbf{x}), (l', \mathbf{x}')\Big)\Big),$$

where

$$\begin{cases} \mu_{\text{pr}}(0, \mathbf{x}) = \mu_0(\mathbf{x}), \quad \Sigma_{\text{pr}}\Big((0, \mathbf{x}), (0, \mathbf{x}')\Big) = \Sigma_0(\mathbf{x}, \mathbf{x}'), \\ \mu_{\text{pr}}(l, \mathbf{x}) = \mu_0(\mathbf{x}), \quad \Sigma_{\text{pr}}\Big((l, \mathbf{x}), (l', \mathbf{x}')\Big) = \Sigma_0(\mathbf{x}, \mathbf{x}') + \mathbb{1}_{\{l=l'\}}\, \Sigma_l(\mathbf{x}, \mathbf{x}'), \quad l, l' = 1, \dots, K. \end{cases}$$

Setting the same prior mean for all models reflects the assumption that all information sources share the same global trend, while discrepancies are captured through additional covariance terms.

In this study, the hyperparameters for the multi-fidelity GP metamodel are estimated using MLE[10]. The mean function is assumed to be constant, which corresponds to an ordinary Kriging model, while isotropic Gaussian covariance kernels are considered. In what follows, the covariance kernels are expressed as the product of a marginal variance parameter σ_l^2, $l = 0, \dots K$, and a correlation kernel, the latter encoding the spatial dependence structure of the process. Specifically, the covariance kernel of the high-fidelity Gaussian process is defined by

$$\Sigma_0(\mathbf{x}, \mathbf{x}') = \sigma_0^2 \exp\left(-\sum_{i=1}^{d} \frac{(x_i - x_i')^2}{2\, h_0^2}\right),$$

and, for each low-fidelity level $l = 1, \dots, K$, the discrepancy process is modeled using a Gaussian kernel given by

$$\Sigma_l(\mathbf{x}, \mathbf{x}') = \sigma_l^2 \exp\left(-\sum_{i=1}^{d} \frac{(x_i - x_i')^2}{2\, h_l^2}\right),$$

where, respectively for the high-fidelity and the low-fidelity models, h_0, h_l denote the processes hyperparameters and σ_0^2, σ_l^2 are the processes marginal variance.

5.1.2 The training point selection procedure

According to **section 3.1**, the posterior distribution for a new input $\mathbf{x}^*$ and information source $l = 0, ...K$, given the set of observations $\mathcal{D}_n$, is Gaussian and defined as:

$$G_P(l, \mathbf{x}^*) \mid \mathcal{D}_n \sim \mathcal{N}\Big(\mu_P(l, \mathbf{x}^*), \sigma_P^2(l, \mathbf{x}^*)\Big). \tag{10}$$

where $\mu_P(l, \mathbf{x}^*)$ denotes the posterior predictive mean and $\sigma_P^2(l, \mathbf{x}^*)$ the posterior predictive variance of the metamodel distribution at $\mathbf{x}^*$. These two quantities are computed using the expressions given in Equations (5). In the present work, all information sources are assumed to be deterministic. Consequently, no observation noise is considered in the Gaussian process formulation. As a result, the Gaussian process interpolates the training data exactly, and the posterior predictive variance vanishes at the training points.

We next describe the approach used to select and add a future training point $\mathbf{x}^{n+1}$. Let y^F denote a hypothetical future simulated response at the candidate input $\mathbf{x}^{n+1}$ from the information source l_F, such

[10]The corresponding hyperparameters are estimated by maximizing the log-likelihood function associated with the GP model. The explicit formulation of the likelihood function and the associated hyperparameter estimation procedure are detailed in Appendix A.2.

that $y^F = G_P(l_F, \mathbf{x}^{n+1})$. Conditioned on the current set of observations $\mathcal{D}_n$, the random variable y^F follows the posterior predictive distribution of the Gaussian process:

$$y^F \mid \mathcal{D}_n \sim \mathcal{N}\Big(\mu_P(l_F, \mathbf{x}^{n+1}), \sigma_P^2(l_F, \mathbf{x}^{n+1})\Big).$$

Thanks to this result, one can now show that the posterior distribution of the Gaussian process after adding a new training point $(l_F, \mathbf{x}^{n+1})$ remains Gaussian.

$$G_F\Big(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F\Big) \sim \mathcal{N}\Big(\mu_F\Big(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F\Big), \sigma_F^2\Big(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F\Big)\Big). \tag{11}$$

where $\mu_F(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F)$ and $\sigma_F^2(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F)$ denote respectively the posterior mean and posterior variance of the high-fidelity Gaussian process at $\mathbf{x}$, conditioned on the hypothetical addition of a new training point $\mathbf{x}^{n+1}$ from the information source l_F with response y_F. This result is important, because it gives us the GP distribution that is obtained after adding the new point to the DoE, but without explicitly adding it.

According to Equation (5), the posterior variance of the hypothetical future GP metamodel depends only on the candidate location $\mathbf{x}^{n+1}$ and the information source l_F. Therefore, it can be written as

$$\sigma_F^2\Big(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F\Big).$$

Moreover, since the posterior mean of the hypothetical future high-fidelity GP is affine with respect to the random variable y_F, it is itself a Gaussian random variable. More precisely,

$$\mu_F\Big(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F\Big) \sim \mathcal{N}\Big(\mu_P(0, \mathbf{x}), \bar{\sigma}^2\Big(\mathbf{x} \mid \mathbf{x}^{n+1}, l_F\Big)\Big), \tag{12}$$

where

$$\bar{\sigma}^2\Big(\mathbf{x} \mid \mathbf{x}^{n+1}, l_F\Big) = \frac{\Sigma_P^2((0, \mathbf{x}), (l_F, \mathbf{x}^{n+1}))}{\sigma_P^2(l_F, \mathbf{x}^{n+1})}.$$

Thereby, the role of $\mu_F(0, \mathbf{x} \mid \mathbf{x}^{n+1}, l_F, y_F)$ can be replaced by the current posterior mean $\mu_P(0, \mathbf{x})$.

Finally, this method allows us to compare various data points and their impact once added in the DoE, without costly calculations.

5.2 Introducing the collective learning function

Regarding the next training point to select, that is the "best" or more informative about the limit state boundary, the state of the art mfEGRA method determined it by solving an optimization problem. Compared to other methods such as the sampling approach, it can generate better points. However, as explained in **sub-section 4.1.2**, it sometimes performs poorly in the later iterations of the algorithm. When doing mfEGRA, this study uses sampling.

In multi-fidelity reliability analysis, not only the training point but also the information source needs to be determined. The low-fidelity data can help to explore the approximate surface of the limit state function and the high-fidelity data can help to improve the accuracy of the metamodel. In the proposed AMGPRA method, the algorithm chooses simultaneously the "best" training point and the information source by using a particular learning function, the **collective learning function (CLF)**, that we want to maximize. It also considers the global impact of the new training point $\mathbf{x}^{n+1}$ on the Monte Carlo population $\mathbf{S}$, that is the average improvement in learning function values of all other N_{MCS} points in $\mathbf{S}$ if $\mathbf{x}^{n+1}$ is added to the set of training points. Its expression is given by:

$$CLF(\mathbf{x}^{n+1}, l_F) = \frac{\sum_{i=1}^{N_{\text{MCS}}} \Big[lf\Big(\mu_P(0, \mathbf{x}^{(i)}),\ \sigma_P^2(0, \mathbf{x}^{(i)})\Big) - lf\Big(\mu_P(0, \mathbf{x}^{(i)}),\ \sigma_F^2(0, \mathbf{x}^{(i)} \mid \mathbf{x}^{n+1}, l_F)\Big) \Big]}{N_{\text{MCS}}}.$$

Here, lf refers to a learning function (*e.g.* the EFF learning function or a modified version of the U learning function), such that an informative point has a high value for lf. Thereby, as most points in $\mathbf{S}$ have a lf value close to 0 and are not very informative, in practice we only take a subset N_c of $\mathbf{S}$ constituted only by the most informative points (common choice being $N_c = 1000$).

Finally, as a low-fidelity training point is often significantly cheaper to measure, but with less impact than a high-fidelity training point, we normalize by the cost c_l of the selected information source l:

$$CLF(\mathbf{x}^{n+1}, l_F) = \frac{\sum_{i=1}^{N_c} \left[lf\Big(\mu_P(0, \mathbf{x}^{(i)}),\, \sigma_P^2(0, \mathbf{x}^{(i)})\Big) - lf\Big(\mu_P(0, \mathbf{x}^{(i)}),\, \sigma_F^2(0, \mathbf{x}^{(i)} \mid \mathbf{x}^{n+1}, l_F)\Big) \right]}{N_c\, c_l(\mathbf{x}^{n+1})}.$$

As the CLF considers the global impact of adding a training point from an information source, it should help construct the metamodel more efficiently than the mfEGRA approach, where the training point and information source are identified separately. In addition, due to the flexibility of CLF, it can be easily extended with any lf that fits the profile to adapt to different problems.

5.3 The AMGPRA procedure

The previous results are used in the AMGPRA procedure in order to produce the **Algorithm 1**[11]. While the main concepts have been discussed above, this section provides a detailed explanation of the steps followed by the active learning procedure.

The algorithm is initialized with a Monte Carlo population of size N_{MCS}, generated in the input space $\mathcal{X} = \mathbb{R}^d$ by LHS, and denoted $\mathbf{S}$. N_i points[12] are then randomly selected among $\mathbf{S}$ to evaluate the $K+1$ fidelity models, in order to build the DoE. The initial set of training points $\mathcal{X}_0$ is therefore given by:

$$\mathcal{X}_0 = \{\mathbf{x}^j, l^j\}_{j=1}^{(K+1)N_i} \tag{13}$$

Let $n = (K+1)N_i$. Once the algorithm is initialized, the multi-fidelity GP metamodel is build according to the description provided in **sub-section 5.1**, leading to the estimates $\hat{g}(l, \mathbf{x}|\{\mathbf{x}^j, l^j\}_{j=1}^n)$, which are defined by the values for the model's parameters estimated by MLE. The high-fidelity estimate $\hat{g}(0, \mathbf{x}|\{\mathbf{x}^j, l^j\}_{j=1}^n)$ is computed for every point in $\mathbf{S}$ (even the points in the DoE), and outputs posterior mean $\mu_P(0, \mathbf{x}^*)$ and posterior variance $\sigma_P^2(0, \mathbf{x}^*)$. Then, according to $\mu_P(0, \mathbf{x}^*)$, the probability of failure $\hat{P}_f$ is computed by MCS, taking the empirical mean on the Monte Carlo population of all the points that fail under the failure boundary, according to Equation (2). A first stopping criterion for the addition of new points in the DoE is checked, based on the EFF function from Equation (7), and is called the *maximum error*:

$$Err_{\max} = \max(EFF) < 0.001 \tag{14}$$

If this criterion is satisfied, the process leads to a second stopping criterion consisting of checking if the coefficient of variation (*i.e.* the ratio of the standard deviation over the mean) of the failure probability is below a predefined threshold of 5%:

$$COV_{\widehat{P}_f} = \sqrt{\frac{1 - \widehat{P}_f}{N_{MCS}\widehat{P}_f}} < 0.05 \tag{15}$$

If this criterion is not satisfied, new generated data points $\Delta\mathbf{S}$ are added in $\mathbf{S}$, and the process go back to the GP metamodel prediction step. Otherwise, the algorithm ends by returning $\widehat{P}_f$.

[11] To the best of our knowledge, the studied procedure does not have a publicly accessed algorithm. This section provides an insight on how such a process can be developed as an algorithm, which clearly constitutes a further development for our work.

[12] According to the authors, the integer N_i is selected thanks to the formula: $\min\left\{12,\ \frac{(d+1)(d+2)}{2}\right\}$, with d the number of input variables.

Now, if the first criterion was not satisfied, N_c points from $\mathbf{S}$ are chosen with the largest value of the learning function lf. The CLF process detailed in **sub-section 5.2** is therefore applied to determine the next training point and the information source to add in the DoE. If the information source selected is high-fidelity, the training point from all the fidelities are added to the training set instead of just the high-fidelity one. The set of training points is then updated, and the multi-fidelity metamodel is retrained with this augmented set. The procedure loops back to the prediction step until both criteria are satisfied. Finally, the value $\hat{P}_f$ is reported as the estimation of the probability of failure.

Algorithm 1: AMGPRA procedure

Input: Initial DoE $\mathcal{X}_0$, Monte Carlo sample size N_{MCS}
Output: Estimated probability of failure $\hat{P}_f$

```
procedure AMGPRA(X_0):
    X ← X_0, Generate S, Train multi-fidelity GP ĝ on X
    while True do
        Compute P̂_f, Err_max and COV_P̂f on S using ĝ
        if Err_max ≥ 0.001 then
            Select (x_new, l_new) via the CLF
            if l_new = 0 then
                Add x_new to all fidelities in X
            Retrain ĝ on updated X
        else if COV_P̂f > 0.05 then
            Generate ΔS and update S ← S ∪ ΔS
        else
            return P̂_f
```

5.4 Application: the AMGPRA method in practice

The benefits of the AMGPRA approach can be assessed on a benchmark example. The proposed benchmark problem is widely used in the reliability literature[13] and is considered as a representative test case.

Let x_1 and x_2 being two input variables, such that $x_1 \sim \mathcal{N}(1.5, 1)$ and $x_2 \sim \mathcal{N}(2.5, 1)$. The limit state function is taken to be multi-modal and nonlinear, and is defined as follows:

$$g_0(x_1, x_2) = \frac{(x_1^2 + 4)(x_2^2 - 1)}{20} - \sin\left(\frac{5x_1}{2}\right) - 2 \tag{16}$$

g_0 is considered as the high-fidelity model, *i.e.* the ground truth, and coincides exactly with the true limit state function. This generates a failure surface that is highly irregular, as shown in Figure 2, which makes the problem significantly more difficult than standard toy examples. Such complexity is important because multi-fidelity strategies must decide correctly where and which model to evaluate to explore the whole region efficiently. The high-fidelity model g_0 is then assumed to coincide exactly with the true limit-state function. Two low-fidelity models are also considered in this problem:

$$g_1(x_1, x_2) = g_0(x_1, x_2) + \sin\left(\frac{5x_1}{22} + \frac{5x_2}{44} + \frac{5}{4}\right) \tag{17}$$

$$g_2(x_1, x_2) = g_0(x_1, x_2) + 3\sin\left(\frac{5x_1}{11} + \frac{5x_2}{11} + \frac{35}{11}\right) \tag{18}$$

Each of these three models has an associated cost reflecting the difficulty of evaluating the model at a new training point: $c_0 = 1$, $c_1 = 0.1$, and $c_2 = 0.01$.

[13] Introduced in [18].

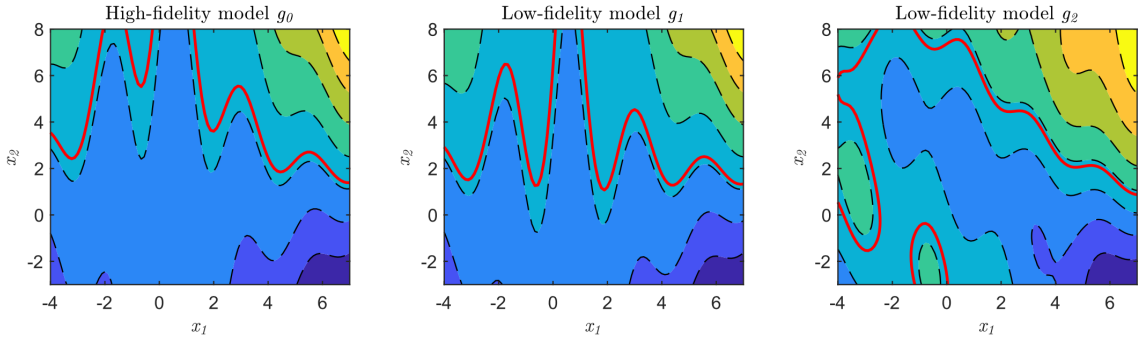

Figure 2: Contours of the high-fidelity and low-fidelity models. The red curves represent the limit-state surfaces $g_l(\mathbf{x}) = 0$. Adapted from Zhang et al. (2022) [1].

Then, the probability of failure is first estimated using a Monte Carlo sampling performed solely with the high-fidelity model. The estimate is obtained from 10^6 samples and is used as the reference failure probability. Several surrogate-based reliability methods are then tested and compared, including the proposed method and other widely used approaches mentionned above, such as AK-MCS-EFF and mfEGRA. For all methods, the initial population $\mathbf{S}$ has size $N_{\text{MCS}} = 10^4$. Since we work with 2 input variables, only 6 points are initially selected in the DoE.

A tri-fidelity case[14] is then considered, involving all the models g_0, g_1, and g_2. The different methods are compared in terms of the average computational cost, the average estimated probability of failure, and the average relative error (ARE). The ARE measures the average percentage deviation from the reference probability of failure:

$$\text{ARE} = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{P_f^{\text{ref}} - \hat{P}_f(i)}{P_f^{\text{ref}}}\right|, \qquad N = \text{number of repetitions.}$$

All computations are repeated 20 times, and the reported values correspond to the averages over these repetitions. The results are presented below.

Table 1: Results with two low-fidelity models

Method	**Average cost**	**Average $\hat{P}_f$**	**Average relative error (%)**
MCS (reference)	10^6	0.0313	-
AK-MCS-EFF	45.2	0.0313	0.04
AMK-MCS-EFF	14.31	0.0313	0.64
mfEGRA	12.87	0.0311	0.02
AMGPRA-LF-EFF	12.32	0.0310	0.02
AMGPRA-LF-Um	12.41	0.0312	0.03

These results show that both AMGPRA variants achieve the lowest computational cost among all methods. In particular, AMGPRA-LF-EFF reaches an average relative error comparable to the state-of-the-art mfEGRA, while requiring slightly fewer computational resources.

Overall, this example illustrates the ability of the AMGPRA framework to efficiently exploit multiple fidelity levels, achieving accurate failure probability estimates at a significantly reduced computational cost.

[14]In the original paper, a bi-fidelity setting involving g_0 and a g_1 is also presented. AMGPRA outperforms competing methods in terms of computational cost and accuracy on this case, but results are not shown for brevity.

6 Implementation and experimental framework

This section describes the practical realisation of the two multi-fidelity active learning algorithms studied in this project, `mfEGRA` and `AMGPRA`. Since neither implementation is publicly available, both were reconstructed from scratch on the basis of their respective reference papers, using the `SMT` Python library as the underlying surrogate engine.

As this report focuses on multi-fidelity models, we will present only the implementations for the multi-fidelity methods. Nevertheless, we also successfully implemented the `AK-MCS` method, and details are presented in the Appendix **sub-section A.3**.

6.1 General implementation framework

Given that the code used by the authors of the AMGPRA method is not available online, it was necessary to reconstruct the entire algorithm presented in the article. To do so, we were required to rely on the Python library `SMT` (*Surrogate Modeling Toolbox*).

The implementation developed in this work relies on the `MFK` (*Multi-Fidelity Kriging*) model available in the `SMT` library. This model is a multi-fidelity Gaussian process approach based on an additive first-order autoregressive formulation, introduced by Kennedy and O'Hagan [6]. In the two-level case, the high-fidelity response is modeled as

$$Z_{\mathrm{HF}}(\mathbf{x}) = \rho(\mathbf{x})\, Z_{\mathrm{LF}}(\mathbf{x}) + \delta(\mathbf{x}), \tag{19}$$

where $Z_{\mathrm{LF}}(\mathbf{x})$ denotes the low-fidelity Gaussian process, $\rho(\mathbf{x})$ is a scaling function, and $\delta(\mathbf{x})$ is an additional Gaussian process accounting for the discrepancy between the two fidelity levels. The adjustment function $\rho(\mathbf{x})$ may be chosen as constant, linear, or quadratic depending on the level of flexibility required by the model.

The implementation provided in `SMT` follows the recursive formulation proposed by Le Gratiet [7], which facilitates the extension to several fidelity levels and improves computational efficiency for larger training sets. In our work, this model serves as the main surrogate modeling tool used to combine information from simulations performed at different fidelity levels.

6.2 mfEGRA implementation

This subsection presents the main steps of our mfEGRA implementation and clarifies how the different components of the original method are approximated and evaluated in practice.

Computation of EFF At each iteration, the Expected Feasibility Function (EFF) is evaluated over the whole candidate set

$$S = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N_{\mathrm{MC}})}\}.$$

In the implementation, the high-fidelity predictions are obtained from the `MFK` model of the `SMT` library through the functions `predict_values` and `predict_variances`. More precisely, for all points $\mathbf{x} \in S$, the posterior mean is computed with

$$\mu_P(0, \mathbf{x}) = \texttt{self.model.predict_values}(\mathbf{x}),$$

while the posterior variance is computed with

$$\sigma_P^2(0, \mathbf{x}) = \texttt{self.model.predict_variances}(\mathbf{x}).$$

In the code, these two quantities are retrieved through the wrapper function `predict_hf(self.S)`, which returns the vectors of posterior means and standard deviations over the whole population S. The EFF is then evaluated at every candidate point using

$$\mathrm{EFF}(\mathbf{x}) = \mathrm{EFF}\Big(\mu_P(0,\mathbf{x}), \sigma_P(0,\mathbf{x})\Big), \qquad \sigma_P(0,\mathbf{x}) = \sqrt{\sigma_P^2(0,\mathbf{x})},$$

with the feasibility band chosen as $\varepsilon(\mathbf{x}) = 2\,\sigma_P(0,\mathbf{x})$. Therefore, the enrichment point is selected by scanning the whole set S and taking

$$\mathbf{x}_{n+1} \approx \arg\max_{\mathbf{x}\in S} \mathrm{EFF}(\mathbf{x}).$$

In the present implementation, the EFF is evaluated over the whole candidate set S, so that the next enrichment point is selected by exhaustive search within this discrete population. Since the original method already relies on a discretization of the input space, the possible improvement would not have been to replace S by a continuous optimization procedure, but rather to reduce the number of candidate points actually screened inside S. For example, one could have considered adaptive or partial-search strategies. Nevertheless, the full scan of S was preferred here in order to keep the implementation simple, robust, and fully consistent with the discrete Monte Carlo framework.

Computation of DL Once the next enrichment point $\mathbf{x}_{n+1}$ has been selected, the next fidelity level is chosen by maximizing a discrete approximation of the weighted look-ahead information gain over the available sources. In the present implementation, this criterion is not evaluated over the whole candidate population S, but over a reduced subset

$$Z_{N_c} \subset S,$$

containing the N_c points with the largest EFF values. For each candidate fidelity level l, the score is computed as

$$J_l(\mathbf{x}_{n+1}) \approx \frac{1}{c_l} \sum_{\mathbf{z}\in Z_{N_c}} \mathrm{EFF}(\mathbf{z})\, D_l(\mathbf{z} \mid \mathbf{x}_{n+1}), \tag{20}$$

where c_l denotes the cost associated with fidelity level l. In the code, this quantity is evaluated by the function `_weighted_info_gain_objective_exact_raw`, and then normalized by the corresponding cost.

The use of the truncated set Z_{N_c} is a practical choice made to reduce the computational burden of the source-selection stage. Indeed, evaluating D_l over the whole population S would be significantly more expensive, especially when S contains several tens of thousands of candidate points. Restricting the computation to the points with the largest EFF values makes it possible to focus the calculation on the most informative region of the candidate set. If larger computational resources were available, the same criterion could in principle be evaluated over the whole set S.

More precisely, for each $\mathbf{z} \in Z_{N_c}$, the local divergence term is defined by

$$D_l(\mathbf{z} \mid \mathbf{x}_{n+1}) = \log\left(\frac{\sigma_F(0,\mathbf{z} \mid \mathbf{x}_{n+1}, l)}{\sigma_P(0,\mathbf{z})}\right) + \frac{\sigma_P^2(0,\mathbf{z}) + \bar{\sigma}^2(\mathbf{z} \mid \mathbf{x}_{n+1}, l)}{2\,\sigma_F^2(0,\mathbf{z} \mid \mathbf{x}_{n+1}, l)} - \frac{1}{2}. \tag{21}$$

In the implementation, all covariance quantities entering this expression are reconstructed through the function `_post_cov`. This function computes the posterior covariance by the standard Gaussian conditioning formula:

$$\Sigma_P\Big((l,\mathbf{x}),(l',\mathbf{x}')\Big) = \Sigma_{\mathrm{pr}}\Big((l,\mathbf{x}),(l',\mathbf{x}')\Big) - \Sigma_{\mathrm{pr}}\Big((l,\mathbf{x}),\mathcal{D}\Big)\,\Sigma_{\mathrm{pr}}(\mathcal{D},\mathcal{D})^{-1}\,\Sigma_{\mathrm{pr}}\Big(\mathcal{D},(l',\mathbf{x}')\Big), \tag{22}$$

where $\mathcal{D}$ denotes the current multi-fidelity training set. In the code, the matrix $\Sigma_{\mathrm{pr}}(\mathcal{D},\mathcal{D})$ is factorized once by Cholesky decomposition and reused in the subsequent covariance evaluations.

Using this posterior covariance operator, the present posterior variance of the high-fidelity model at $\mathbf{z}$ is obtained as

$$\sigma_P^2(0,\mathbf{z}) = \mathrm{diag}\Big(\texttt{_post_cov}(0, Z_{N_c}, 0, Z_{N_c})\Big), \tag{23}$$

the posterior covariance between the high-fidelity response at $\mathbf{z}$ and the response of fidelity l at $\mathbf{x}_{n+1}$ is obtained as

$$\mathrm{Cov}_P\Big(\hat{g}_0(\mathbf{z}), \hat{g}_l(\mathbf{x}_{n+1})\Big) = \texttt{_post_cov}(0, Z_{N_c}, l, \mathbf{x}_{n+1}), \tag{24}$$

and the posterior variance at the candidate point $\mathbf{x}_{n+1}$ for fidelity l is computed as

$$\mathrm{Var}_P\Big(\hat{g}_l(\mathbf{x}_{n+1})\Big) = \texttt{_post_cov}(l, \mathbf{x}_{n+1}, l, \mathbf{x}_{n+1}). \tag{25}$$

From these quantities, the code evaluates

$$\bar{\sigma}^2(\mathbf{z} \mid \mathbf{x}_{n+1}, l) \approx \frac{\mathrm{Cov}_P^2\Big(\hat{g}_0(\mathbf{z}), \hat{g}_l(\mathbf{x}_{n+1})\Big)}{\mathrm{Var}_P\Big(\hat{g}_l(\mathbf{x}_{n+1})\Big)}, \tag{26}$$

which corresponds to the variable `sigma_bar2` in the implementation. The hypothetical future variance is then approximated by

$$\sigma_F^2(0, \mathbf{z} \mid \mathbf{x}_{n+1}, l) \approx \sigma_P^2(0, \mathbf{z}) - \bar{\sigma}^2(\mathbf{z} \mid \mathbf{x}_{n+1}, l), \tag{27}$$

and its square root yields $\sigma_F(0, \mathbf{z} \mid \mathbf{x}_{n+1}, l)$. The value of D_l is finally computed by the function `_D_kl_local` from $\sigma_P(0, \mathbf{z})$, $\sigma_F(0, \mathbf{z} \mid \mathbf{x}_{n+1}, l)$, and $\bar{\sigma}^2(\mathbf{z} \mid \mathbf{x}_{n+1}, l)$.

It should be emphasized that these covariance terms are not extracted directly from the internal posterior covariance operator of the recursive `MFK` model. Instead, they are reconstructed from a simplified additive multi-fidelity covariance structure using the hyperparameters estimated by `SMT`, namely the kernel parameters θ_l and the process variances σ_l^2.

Finally, a heuristic correction is introduced for the high-fidelity source $l = 0$. In practice, without additional penalization, the raw source-selection criterion tended to select the high-fidelity source too frequently, leading to a behavior close to a mono-fidelity strategy. To mitigate this effect, the variance term used in the denominator of

$$\bar{\sigma}^2(\mathbf{z} \mid \mathbf{x}_{n+1}, l) = \frac{\mathrm{Cov}_P^2\Big(\hat{g}_0(\mathbf{z}), \hat{g}_l(\mathbf{x}_{n+1})\Big)}{\mathrm{Var}_P\Big(\hat{g}_l(\mathbf{x}_{n+1})\Big)}$$

is modified when $l = 0$ by adding a constant offset:

$$\mathrm{Var}_P\Big(\hat{g}_0(\mathbf{x}_{n+1})\Big) \leftarrow \mathrm{Var}_P\Big(\hat{g}_0(\mathbf{x}_{n+1})\Big) + 0.3. \tag{28}$$

This modification decreases the corresponding value of $\bar{\sigma}^2$, reduces the estimated look-ahead variance reduction associated with the high-fidelity source, and therefore penalizes its selection. This term should not be interpreted as part of the original mfEGRA formulation, but rather as an empirical rebalancing heuristic introduced in the implementation to encourage a more effective use of the lower-fidelity models.

UML diagram of the MFEGRA implementation Figure 15 summarizes the software architecture of the proposed implementation. The central component is the `MFEGRA` class, which acts as the main orchestration layer of the algorithm. It stores the list of limit-state functions associated with the different fidelity levels, as well as their respective evaluation costs, and manages the overall adaptive enrichment process through methods such as `run()` and `enrich()`. In particular, this class is responsible for driving the sequential strategy, rebuilding the covariance cache, and evaluating the weighted information gain criterion used to select the next fidelity level.

The `MFEGRA` class relies on the auxiliary class `MultiFidelityModel`, which provides an abstraction layer over the `MFK` implementation of the `SMT` library. This wrapper encapsulates the calls to the external surrogate model, handles training through the method `train(xt, yt)`, and exposes prediction and parameter-retrieval routines such as `predict_hf(x)` and `get_sigma2_for_fid(fid)`. In this way, the

details specific to the `SMT` interface remain isolated from the higher-level logic of the adaptive algorithm.

The UML diagram also highlights the role of the `AcquisitionLogic` utility block, which represents the mathematical criteria used during enrichment, namely the Expected Feasibility Function (EFF) and the Kullback–Leibler-based look-ahead information gain. Even though these criteria are implemented through dedicated functions rather than through a fully separate class in the code, they form a distinct logical layer within the architecture. Overall, this organization promotes a clear separation of concerns between surrogate-model management, acquisition-criterion evaluation, and sequential decision-making, which improves both the readability and the extensibility of the implementation.

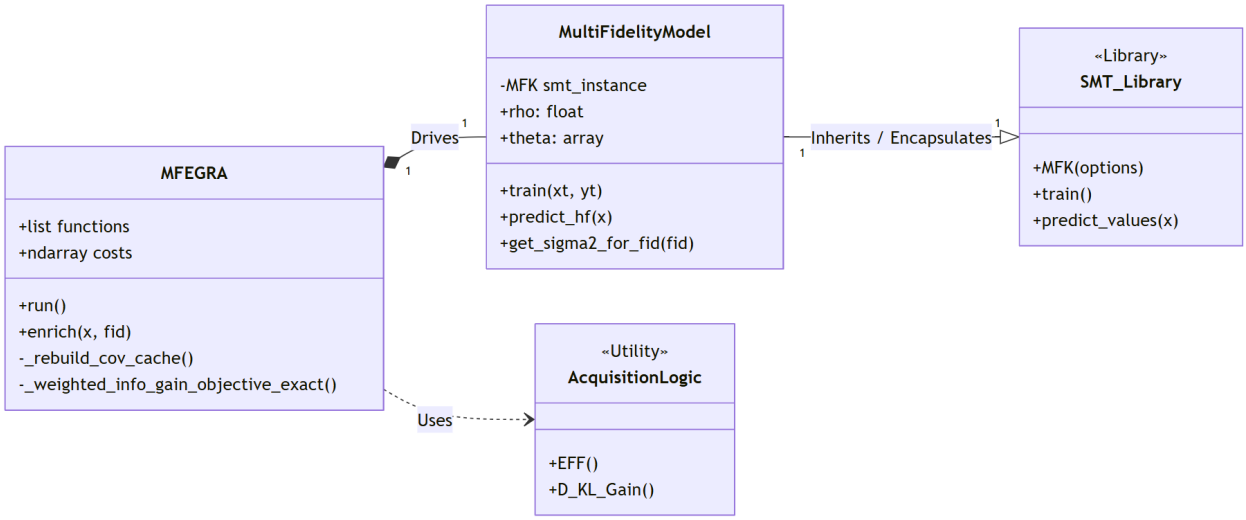

Figure 3: UML class diagram of the MFEGRA implementation.

6.3 AMGPRA implementation

The implementation of the AMGPRA strategy follows the same global sequential enrichment philosophy as mfEGRA, but with a simplified acquisition mechanism that directly couples the EFF with a cost-aware source selection. From a software perspective, the algorithm is structured around three main components: (i) the orchestration class `AMGPRA`, (ii) a surrogate wrapper `MultiFidelityModel`, and (iii) a set of auxiliary learning functions gathered in `LearningFunctions`. This modular design reflects the separation between the mathematical formulation of the method and its practical implementation.

At each iteration, the current multi-fidelity surrogate is queried over a Monte Carlo candidate set

$$S = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N_{\mathrm{MC}})}\},$$

using the wrapper `MultiFidelityModel`, which internally relies on the `MFK` model from the SMT library. In practice, this wrapper isolates all interactions with the external surrogate: the methods `train` and `predict` respectively handle the construction of the model and the evaluation of the posterior mean and variance. As in mfEGRA, the quantities

$$\mu_P(0, \mathbf{x}) \quad \text{and} \quad \sigma_P^2(0, \mathbf{x})$$

are obtained through calls to `predict_values` and `predict_variances`, but the implementation deliberately avoids exposing the internal structure of the recursive multi-fidelity model to the higher-level algorithm.

The exploration of the input space is driven by the EFF, implemented in the utility block `LearningFunctions` Given the posterior statistics, the EFF is evaluated pointwise as a function of (μ_P, σ_P), and the enrichment point is selected through a discrete maximization over S. This step closely mirrors the mathematical definition, but an important practical aspect is that all computations are vectorized at the code level,

allowing the full candidate set to be processed efficiently despite its large size.

Once the next candidate point $\mathbf{x}_{n+1}$ has been identified, the originality of AMGPRA lies in the way the fidelity level is selected. Instead of relying on an explicit look-ahead information gain involving full posterior covariance updates (as in mfEGRA), the method uses a proxy criterion implemented in the function `_compute_clf`. In practice, this avoids the need to explicitly simulate the updated Gaussian process, which would require repeated conditioning operations.

This design choice is directly motivated by one of the main computational difficulties of Gaussian process-based sequential methods, namely the manipulation of posterior covariance operators. In a full look-ahead setting, one needs to evaluate expressions of the form

$$\Sigma_P(\cdot,\cdot) - \frac{\Sigma_P(\cdot,x)\Sigma_P(x,\cdot)}{\Sigma_P(x,x)},$$

which involves extracting cross-covariances between arbitrary sets of points and candidate observations. While this is theoretically straightforward, it becomes cumbersome in practice when using the `MFK` implementation. Moreover, the recursive structure of the multi-fidelity model means that the covariance depends on both the fidelity levels and the hierarchical correlations, making the extraction of consistent sub-blocks non-trivial.

To circumvent these limitations, the AMGPRA implementation does not attempt to reconstruct the full posterior covariance from the kernel hyperparameters (such as the length-scales θ_l and process variances). Instead, it relies exclusively on quantities that are robustly provided by the SMT interface, namely marginal variances and mean predictions. This significantly simplifies the implementation and improves numerical stability, at the cost of using a more heuristic approximation of the expected information gain.

From a software architecture viewpoint, the class `AMGPRA` encapsulates the entire adaptive loop through the methods `run()` and `enrich()`. It stores the list of limit-state functions, their associated costs, and the current dataset. The method `run()` initializes the Design of Experiments via `initialize_doe()`, then iteratively calls `enrich()`, which performs three main steps: (i) evaluation of the EFF over S, (ii) selection of the next point $\mathbf{x}_{n+1}$, and (iii) selection of the fidelity level via `_compute_clf`. The selected observation is then added to the dataset, and the surrogate is retrained.

Overall, the implementation can be seen as a pragmatic translation of the theoretical framework into a numerically efficient algorithm. The key trade-off lies in the handling of covariance information: while the mathematical formulation naturally involves full Gaussian conditioning, the code deliberately restricts itself to quantities that are directly accessible from the surrogate model. This choice ensures a good balance between fidelity to the theory, computational tractability, and robustness of the implementation.

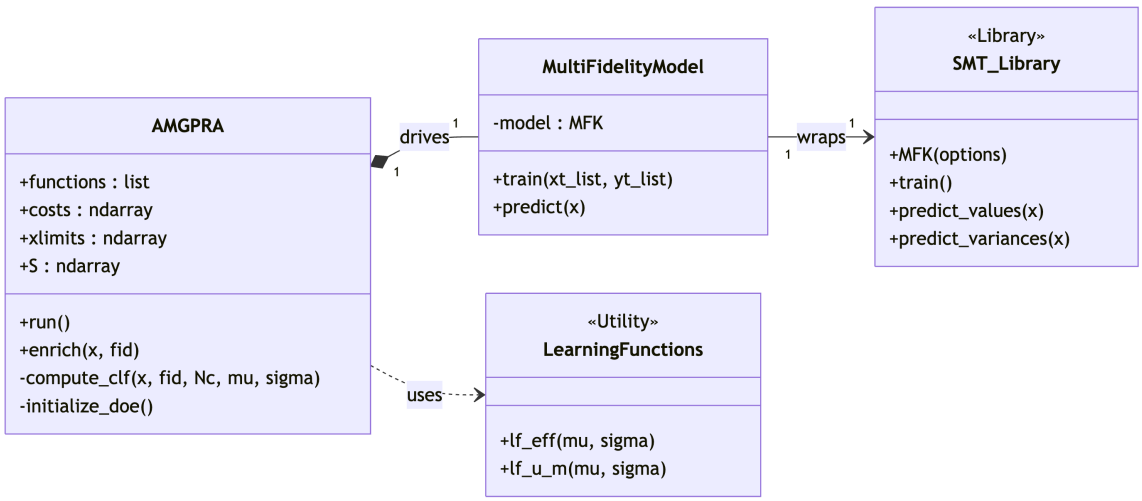

Figure 4: UML class diagram of the AMGPRA method

7 Results and applications

7.1 Reproducibility: testing our implementations on a reference example

In this subsection, we focus on reproducing the results of Example 5.1 from Zhang et al. (2022) [1], presented in Section 5.4. We report the results obtained with our respective implementations of AMGPRA and mfEGRA. We then investigate a modified version of this example in order to assess the sensitivity of AMGPRA and mfEGRA to discrepancies between fidelity levels. More specifically, we consider low-fidelity models that are no longer obtained through the simple additive corrections used in the original example and whose zero contours no longer coincide exactly with that of the high-fidelity model. This modified setting remains informative, since the low-fidelity responses are still correlated with the high-fidelity one, but it provides a biased indication of the failure boundary. It therefore offers a useful test case for evaluating the robustness and efficiency of the adaptive multifidelity strategy when the relationship between fidelity levels departs from the structure considered in the original formulation.

7.1.1 Numerical study based on Example 5.1

We first aim to reproduce exactly Example 5.1, presented in Section 5.4. The reference failure probability is the value obtained by Monte Carlo simulation in the article, namely $\hat{P}_f = 0.0313$. We then carried out five independent runs of our implementations of mfEGRA and AMGPRA. To make the comparison with the article as consistent as possible, we used the same parameter values and the same convergence criteria as those reported by the authors. The corresponding results are summarized in Table 2.

Figures 5 and 6 illustrate the approximated limit-state boundaries obtained with AMGPRA and mfEGRA during a representative run of each algorithm. Overall, both methods capture the main features of the true failure surface with good accuracy. In addition, the number of iterations required for convergence remains moderate, which indicates that both adaptive procedures are able to identify the relevant regions of the input space efficiently.

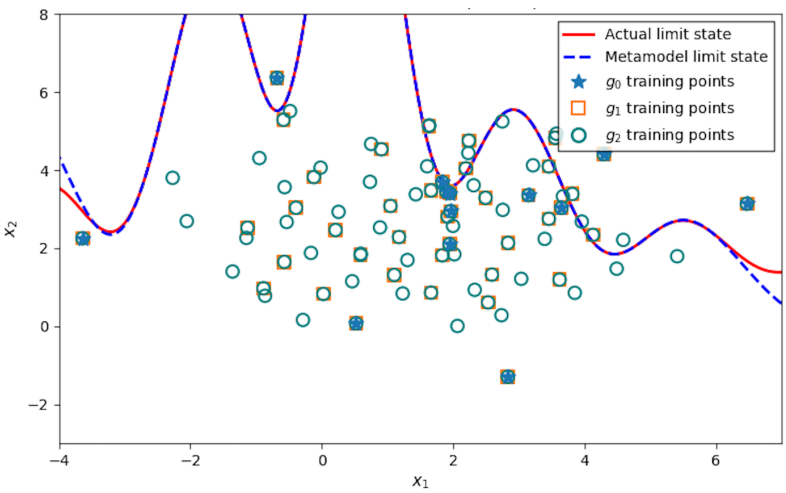

Figure 5: AMGPRA-LF-EFF performance on the 5.1 reference example (converged at iteration 62 on the 5th run).

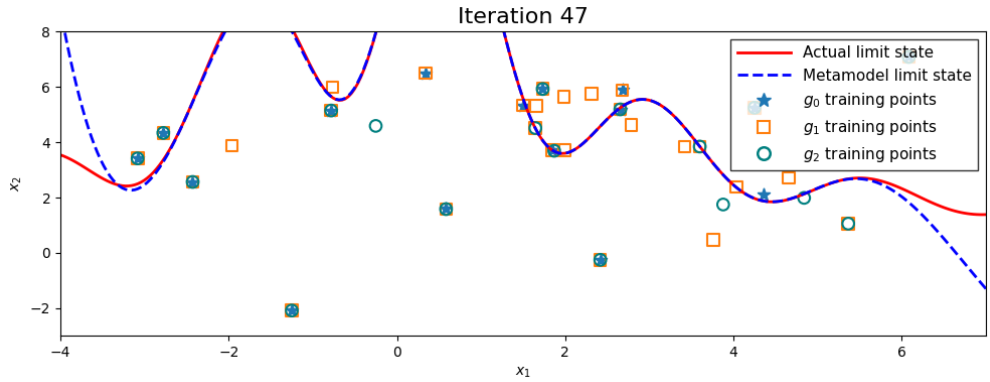

Figure 6: mfEGRA performance on the 5.1 reference example (converged at iteration 47 on the 5th run).

The results reported in Table 2 are globally consistent with those given in the AMGPRA article for the three-fidelity multimodal benchmark. In the reference paper, the authors report average costs of 12.87 for mfEGRA, 12.32 for AMGPRA-LF-EFF, and 12.41 for AMGPRA-LF-Um, with corresponding failure probability estimates of 3.11×10^{-2}, 3.10×10^{-2}, and 3.12×10^{-2}, respectively. The reported average relative errors are 0.02% for mfEGRA and AMGPRA-LF-EFF, and 0.03% for AMGPRA-LF-Um. In our experiments, we obtain average costs of 15.96 for mfEGRA, 15.18 for AMGPRA-LF-EFF, and 14.27 for AMGPRA-LF-Um, together with average failure probability estimates of 0.0312, 0.0313, and 0.0312, and average relative errors of 0.017%, 0.028%, and 0.021%, respectively. These results therefore reproduce the main conclusions of the article: all three methods provide highly accurate estimates of the failure probability at a very low computational cost compared with direct Monte Carlo simulation, and both AMGPRA variants remain slightly less expensive than mfEGRA on this example.

Although our average costs are somewhat higher than those reported in the article, the estimated values of $\hat{P}_f$ remain extremely close to the reference ones, which indicates that our implementations reproduce the benchmark satisfactorily. In particular, the same overall cost hierarchy is globally recovered, with AMGPRA being the cheapest method, while mfEGRA remains the most expensive among the three. The remaining discrepancies may be attributed to differences in the initial designs of experiments, in the Gaussian-process implementation, and in the number of repeated runs used to compute the reported averages. They may also stem from the fact that our implementation does not exactly reproduce the original mfEGRA and AMGPRA frameworks, but rather relies on an approximate surrogate modeling strategy based on the autoregressive multi-fidelity model available in the `SMT` library.

Table 2: Results obtained on the reference version of Example 5.1 over five independent runs.

Method	**Average cost**	**Average $\hat{P}_f$**	**Average relative error (%)**
MCS (reference)	10^6	0.0313	-
AK-MCS-EFF	34	0.0313	0.04
mfEGRA	15.96	0.0312	0.017
AMGPRA-LF-EFF	15.18	0.0313	0.028
AMGPRA-LF-Um	14.27	0.0312	0.021

7.1.2 A modified example: shifted low-fidelity zero contours

In order to further analyze the behavior of AMGPRA and mfEGRA, we consider a modified version of Example 5.1. The probabilistic setting is kept unchanged: the two input variables are taken as $x_1 \sim \mathcal{N}(1.5, 1)$ and $x_2 \sim \mathcal{N}(2.5, 1)$. The high-fidelity model is also kept identical to the reference example and is defined by

$$g_0(x_1, x_2) = \frac{(x_1^2 + 4)(x_2^2 - 1)}{20} - \sin\left(\frac{5x_1}{2}\right) - 2. \tag{29}$$

As in the reference configuration, g_0 is regarded as the high-fidelity model, *i.e.* the ground-truth limit-state function. However, unlike the original benchmark, the low-fidelity models are no longer constructed through simple additive corrections of g_0. Instead, we introduce two modified low-fidelity models whose zero contours are shifted with respect to that of the high-fidelity model.

The first modified low-fidelity model is defined as

$$g_1(x_1, x_2) = (g_0(x_1, x_2) + 0.30)\left[1.6 + 0.3\,\sin\left(\frac{5x_1}{22} + \frac{5x_2}{44} + \frac{5}{4}\right)\cos(x_1 + x_2 + 2)\right]. \tag{30}$$

The second modified low-fidelity model is defined as

$$g_2(x_1, x_2) = (g_0(x_1, x_2) - 0.20)\left[1.4 + 0.25\,\sin\left(\frac{5x_1}{11} + \frac{5x_2}{22} + \frac{35}{11}\right)\cos(x_1 + x_2 + 2)\right]. \tag{31}$$

These two low-fidelity models remain correlated with the high-fidelity response, but they no longer provide the exact same failure boundary. Indeed, since the multiplicative factors remain positive over the domain of interest, the zero contour of g_1 is associated with the level set $g_0(x_1, x_2) = -0.30$, while the zero contour of g_2 corresponds to $g_0(x_1, x_2) = 0.20$, instead of the true failure boundary $g_0(x_1, x_2) = 0$. As a result, the low-fidelity models still contain useful information on the global behavior of the system, but they provide a biased indication of the high-fidelity limit-state surface.

Figures 7 and 8 illustrate representative runs of AMGPRA and mfEGRA on this modified benchmark. In both cases, the estimated failure probability remains close to the reference value, which indicates that the multifidelity strategies are still able to identify the most relevant regions of the input space despite the mismatch between fidelity levels. However, the geometric reconstruction of the limit-state surface is noticeably less accurate than in the original reference example. In particular, both metamodels exhibit spurious contours in peripheral regions and visible deviations from the true boundary, especially away from the central part of the domain where most of the probability mass is concentrated.

This observation is consistent with the construction of the modified low-fidelity models. Since their zero contours no longer coincide with the high-fidelity failure boundary, the information provided by the low-fidelity levels becomes biased. As a result, the adaptive learning process remains informative for estimating P_f, but becomes less efficient for reconstructing the full geometry of the limit-state surface. In particular, the two methods still recover the main oscillatory structure of the true boundary in the region that is most relevant for reliability estimation, but they require substantially more enrichment steps and produce less accurate contours in low-probability areas of the input space.

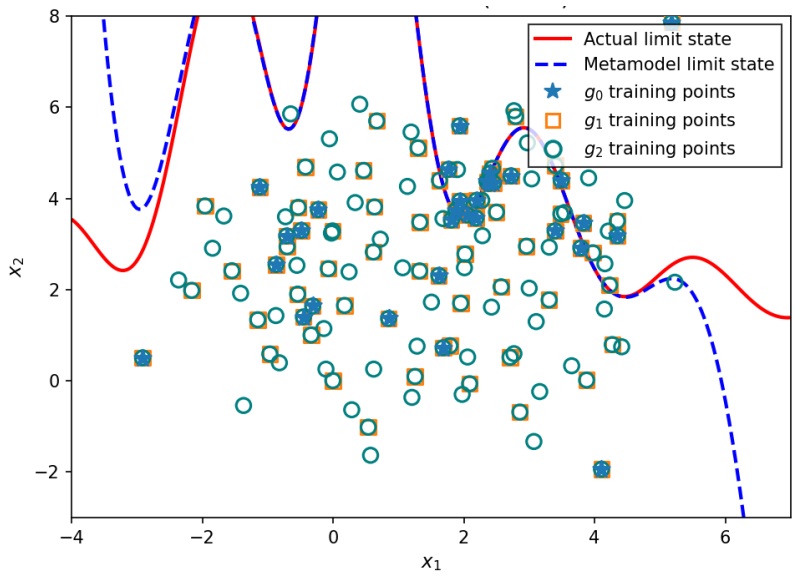

Figure 7: AMGPRA-LF-EFF on the modified reference example (converged at iteration 122 on the 5th run).

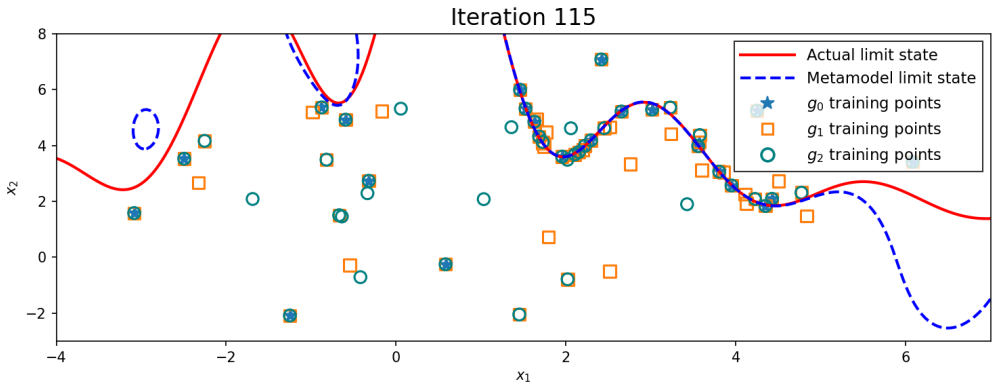

Figure 8: mfEGRA on the modified reference example (converged at iteration 115 on the 5th run).

The quantitative results reported in Table 3 confirm this loss of efficiency. Although all three multifidelity methods still provide accurate estimates of the failure probability, their average computational costs increase significantly compared with the reference example. More precisely, the average cost rises from 15.96 to 33.58 for mfEGRA, from 15.18 to 71.08 for AMGPRA-LF-EFF, and from 14.27 to 72.94 for AMGPRA-LF-Um. Thus, the modified benchmark does not lead to a failure of the methods, but rather to a marked deterioration in their efficiency. In addition, the relative ranking observed on the original example is no longer preserved: whereas the AMGPRA variants were slightly cheaper than mfEGRA in the reference setting, mfEGRA becomes by far the least expensive method on the modified problem.

Overall, these results show that the three methods remain robust in terms of failure probability estimation, since the average values of $\hat{P}_f$ remain very close to the Monte Carlo reference. However, they also highlight a strong sensitivity to the quality of the low-fidelity information. When the low-fidelity zero contours are shifted with respect to the true high-fidelity boundary, the multifidelity strategy still works, but it requires substantially more iterations and a larger computational budget, while the global approximation of the limit-state surface remains imperfect.

Table 3: Results obtained on the modified reference version of Example 5.1.

Method	Average cost	Average $\hat{P}_f$	Average relative error (%)
MCS (reference)	10^6	0.0313	-
mfEGRA	33.58	0.0310	0.021
AMGPRA-LF-EFF	71.08	0.0320	0.045
AMGPRA-LF-Um	72.94	0.0314	0.034

8 Case study: applying AMGPRA to the `solar` benchmark problem

To evaluate the robustness of the AMGPRA method in a real-world industrial setting, we selected the `solar` benchmark [23]. This simulator, developed by the GERAD[15], models a tower-type concentrated solar power (CSP) plant equipped with a molten-salt thermal energy storage system. Its native multi-fidelity interface, high input dimensionality, and expensive evaluations make it a particularly demanding and realistic testbed for our method.

8.1 Physical and numerical context

In order to understand the structure of the problem on which AMGPRA is deployed, we first describe the architecture of the `solar` simulator. The plant relies on four coupled physical subsystems whose numerical modelling constitutes a complex blackbox:

1. **A heliostat field:** An array of sun-tracking mirrors reflects solar radiation toward a receiver at the top of a central tower. The simulator uses a ray-tracing procedure to estimate the delivered power, accounting for atmospheric attenuation and optical spillage losses.
2. **A thermal receiver:** Located at the tower's summit, it absorbs the concentrated solar flux and transfers the resulting heat to molten salt flowing through a network of tubes.
3. **A thermal storage cycle:** Hot molten salt is stored in an insulated tank until needed to drive the power block, enabling continuous electrical production regardless of solar intermittency.
4. **A power block:** A steam generator transfers heat from the molten salt to a Rankine cycle, which drives a turbine to produce electricity.

Figure 9 below provides a schematic overview of the CSP plant cycle, illustrating the molten-salt and steam loops and highlighting the coupling complexity inherent to the simulator.

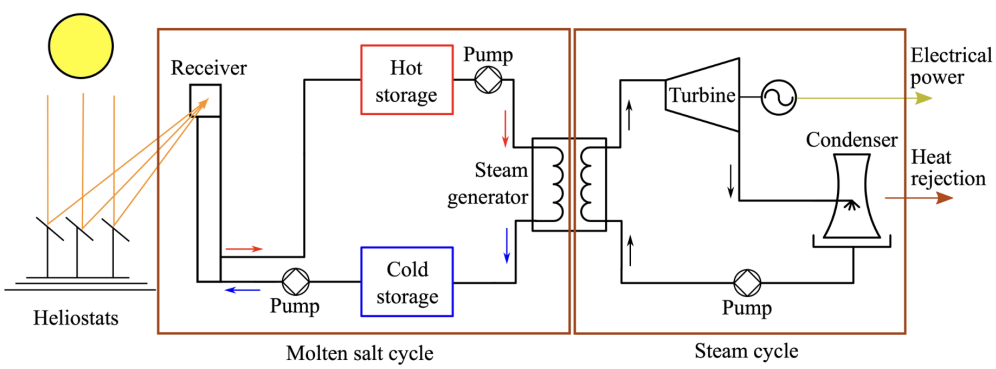

Figure 9: Diagram of the cycle of the concentrated solar power (CSP) plant, from Audet et al.[23].

[15] *Groupe d'études et de recherche en analyse des décisions*, Polytechnique Montréal.

Regarding the dimensionality and blackbox nature of `solar`, the full simulator handles up to 29 decision variables (continuous, discrete, or categorical) and imposes numerous physical constraints. Each function evaluation is computationally expensive, requiring the resolution of transient energy balances and optical radiation algorithms. A key feature for our study is its native multi-fidelity capability: the `-fid` parameter $\phi \in [0, 1]$ allows the user to intentionally degrade the precision of the simulation, reducing the number of rays traced or simplifying the time discretisation, in order to achieve faster computation at the cost of accuracy.

8.2 The `solar 10.1` instance

In order to test our AMGPRA method, we focus on the **solar 10.1** instance, an unconstrained variant of the thermal storage cost minimisation problem (see `solar 6.1` in [23]), selected at the recommendation of our supervisor for its tractability and reduced computational burden.

The input space $\mathcal{X} \subset \mathbb{R}^5$ consists of five continuous variables describing the geometric and thermal characteristics of the storage tanks. The physical search domain is bounded as follows:

- x_1: Hot storage tank height (H_{hot}) $\in$ [1, 50] m
- x_2: Hot storage tank diameter (d_{hot}) $\in$ [1, 30] m
- x_3: Insulation thickness (t_{hot}) $\in$ [0.01, 2] m
- x_4: Receiver outlet temperature ($T_{\text{rcv}}^{\text{out}}$) $\in$ [793, 995] K
- x_5: Minimum cold storage temperature ($T_{\text{cold}}^{\min}$) $\in$ [495, 650] K

The objective is to minimise the cost function $F(\mathbf{x})$, defined through a heuristic penalisation scheme:

$$F(\mathbf{x}) \;=\; h_1(\mathbf{x}) + \frac{10^6}{2}\left[\sum_{i=1}^{6} w_i \cdot g_i(\mathbf{x})^2\right], \tag{32}$$

where $h_1(\mathbf{x})$ is the nominal cost function (covering the material and construction costs), and $g_i(\mathbf{x})$ quantifies violations of the original operational constraints (the thermal stability, budget, and tube pressure). The 10^6 penalisation coefficient creates *cost cliffs* at the boundary of the feasible domain, making F steep and highly nonlinear. For a Gaussian process surrogate, learning this surface requires not only mapping the smooth cost landscape but also implicitly locating the feasibility boundary, a task that makes multi-fidelity active learning particularly valuable.

The reference optimum for this instance, identified by our supervisor using COBYLA[16] (reference run `COBYLA_073` of the `solar 10.1` test set), is:

$$\mathbf{x}_{\text{opt}} \;=\; (994.9999,\ 34.2018,\ 24.2717,\ 0.02449,\ 0.01002), \tag{33}$$

with corresponding optimal value $F(\mathbf{x}_{\text{opt}}) \approx 42$. This reference point serves as the anchor for the reliability analysis formulated below.

[16]**COBYLA** (*Constrained Optimization BY Linear Approximations*) is a derivative-free trust-region optimization algorithm introduced by Powell et al. [24], designed to handle nonlinear constraints without requiring gradient information. At each iteration, it builds a linear approximation of both the objective and the constraint functions within a shrinking trust region, making it well-suited for expensive blackbox simulators such as `solar`.

8.2.1 Transformation into a reliability analysis problem

The core methodological contribution of this project lies in converting the above blackbox optimization task into a structural reliability problem. Rather than simply seeking the minimum of F, we quantify the *robustness* of a design in the neighbourhood of the optimum by asking:

given that $\mathbf{X}$ *is subject to uncertainty, what is the probability that* $F(\mathbf{X})$ *exceeds a given performance threshold* τ*?*

Before formalising this transformation for the five-dimensional `solar` problem, it is instructive to consider the following conceptual illustration. Since $\mathcal{X} \subset \mathbb{R}^5$ cannot be directly visualised, we use a two-dimensional analogue to convey the geometric intuition underlying the limit-state framework.

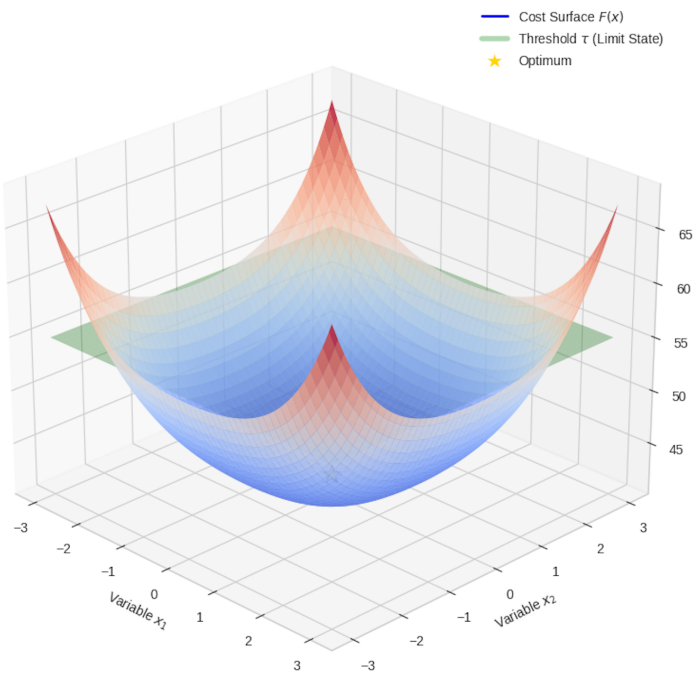

Figure 10: **Conceptual 3D representation of the limit-state surface.** *Note: This surface is an illustrative example and does* ***not*** *represent the high-dimensional* `solar 10.1` *objective function.* The blue bowl-shaped surface is a schematic analogue of a cost function $F(x_1, x_2)$ with a penalised failure region (red spikes). The semi-transparent green plane represents the threshold $\tau = F(\mathbf{x}_{\text{opt}})(1+\alpha)$. Points below the plane satisfy $g(\mathbf{x}) = \tau - F(\mathbf{x}) > 0$ (*safe domain*), while those above it constitute the failure domain. The probability of failure P_f is the probability mass of the input distribution that lies in the failure domain. In five dimensions, the threshold becomes a hyperplane that partitions $\mathcal{X}$ into two regions, whose common boundary is the *limit-state surface* $\{g(\mathbf{x}) = 0\}$.

The key insight conveyed by Figure 10 is that the multi-fidelity Gaussian process surrogate built by AMGPRA must accurately approximate not the entire surface F, but specifically the location of the level set $\{\mathbf{x} : F(\mathbf{x}) = \tau\}$. This is why an active learning procedure that concentrates evaluations near the decision boundary is particularly efficient in this context.

<u>Limit-state function:</u> We define failure as the inability of the system to keep its cost below the threshold τ. The limit-state function is:

$$g(\mathbf{x}) \;=\; \tau - F(\mathbf{x}). \tag{34}$$

The failure event corresponds to $\{g(\mathbf{X}) \leq 0\}$, i.e. to $F(\mathbf{X}) > \tau$. The probability of failure is then: $P_f = \mathbb{P}(F(\mathbf{X}) > \tau)$. It represents the probability that a chosen design (subject to parameter uncertainties) exceeds the allocated budget or violates physical safety constraints.

Threshold and fragility curve: The threshold τ is defined through a *budget overrun parameter* $\alpha \geq 0$:

$$\tau(\alpha) \;=\; F(\mathbf{x}_{\text{opt}}) \times (1+\alpha). \tag{35}$$

By sweeping α over a range of values, one obtains a *fragility curve* $\alpha \mapsto P_f(\alpha)$. As $\alpha \to 0$, the safe zone shrinks to a neighbourhood of the optimum and $P_f \to 1/2$. As $\alpha \to \infty$, almost all of the input distribution falls within the safe zone and $P_f \to 0$. This monotonically decreasing relationship is the object of estimation by AMGPRA. In this study, we evaluate P_f for the following overrun levels:

$$\alpha \;\in\; \{0.1\%,\; 1\%,\; 5\%,\; 10\%,\; 20\%,\; 30\%,\; 60\%\}, \tag{36}$$

covering a wide range from a nearly tight constraint (near-optimum) to a generous safety margin, so as to characterise both the steep and the flat regimes of the fragility curve.

Probabilistic model for the input variables: Following our supervisor's recommendation, the input vector $\mathbf{X}$ is modelled as a multivariate Gaussian distribution centred at the reference optimum $\mathbf{x}_{\text{opt}}$, with a coefficient of variation of 1% applied independently to each component:

$$X_i \;\sim\; \mathcal{N}\Big(x_{\text{opt},\,i},\; \sigma_i^2\Big), \qquad \sigma_i \;=\; 0.01 \times x_{\text{opt},\,i}, \qquad i = 1, \ldots, 5, \tag{37}$$

with the components assumed mutually independent. This choice reflects engineering practice: small parametric uncertainties around a nominal operating point. The 1% coefficient of variation is deliberately modest, which concentrates the probability mass near $\mathbf{x}_{\text{opt}}$ and implies that, for large overrun levels α, the probability of failure will be close to zero, a regime that challenges Monte Carlo-based estimators and makes active surrogate methods like AMGPRA particularly relevant.

8.2.2 Fidelity calibration and computational trade-off

Before running the full AMGPRA pipeline, a preliminary benchmark was conducted to calibrate the bi-fidelity strategy and justify the choice of $\phi_{LF} = 0.5$. Using the `mfEGRA` implementation available in our codebase, we evaluated the `solar 10.1` simulator at four fidelity levels $\phi \in \{0.25, 0.50, 0.75, 1.0\}$, measuring for each level the relative error with respect to the high-fidelity output and the corresponding fraction of the full computing time. The results are displayed in Figure 11.

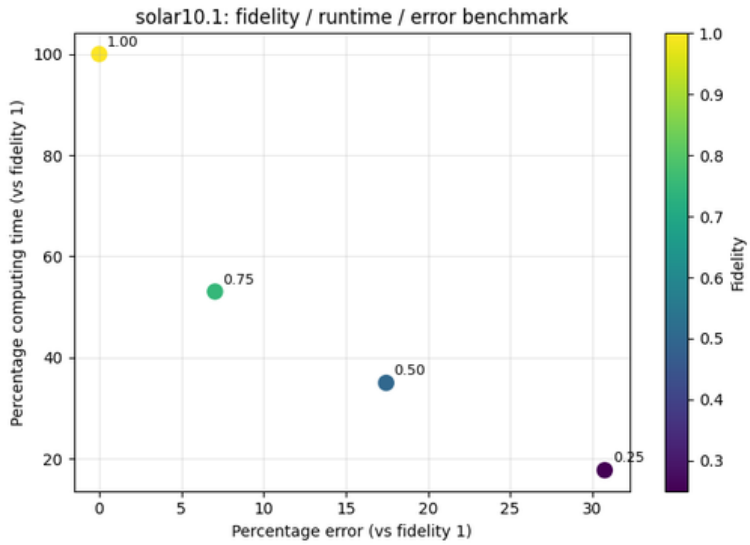

Figure 11: Fidelity/runtime/error trade-off for the `solar 10.1` simulator at four fidelity levels $\phi \in \{0.25, 0.50, 0.75, 1.0\}$. The horizontal axis reports the percentage error relative to full-fidelity output, and the vertical axis reports the percentage of the full-fidelity computing time. The color scale encodes the fidelity level ϕ.

Figure 11 reveals a clear Pareto structure: the level $\phi = 0.75$ reduces computing time by approximately 47% at the cost of a modest 7% error, while $\phi = 0.25$ achieves the greatest speed-up (around 80% faster) but introduces a 31% error that would severely compromise the accuracy of the limit-state prediction. The level $\phi = 0.5$ occupies a favourable intermediate position: it requires only 35% of the full-fidelity budget while incurring an error of approximately 18%, which is acceptable in the exploration phase of AMGPRA where the low-fidelity model is not used to estimate P_f directly, but merely to guide the acquisition of informative points near the boundary $\{g(\mathbf{x}) = 0\}$. On the basis of this benchmark and following our supervisor's recommendation, we adopt the bi-fidelity setup $(\phi_{LF}, \phi_{HF}) = (0.5, 1.0)$ throughout the AMGPRA experiments reported below.

It should be noted that this benchmark was produced using `mfEGRA`, which ran a global reliability analysis over the full design domain with a uniform input distribution, yielding an estimated failure probability of $\hat{P}_f^{\texttt{mfEGRA}} \approx 0.45$ at a total cost of 90.3 equivalent high-fidelity calls over 80 iterations. This result is not directly comparable to the AMGPRA estimates reported in **Section 8.3** below, as the two analyses operate under fundamentally different input distributions. The `mfEGRA` result reflects a *global* sensitivity measure (the fraction of the entire design space that exceeds the cost threshold) whereas AMGPRA targets a *local* robustness analysis centred on $\mathbf{x}_{\text{opt}}$ under the 1% Gaussian uncertainty model of Eq. (37). Both perspectives are complementary: the former characterises the difficulty of the optimisation landscape at large, while the latter quantifies the engineering robustness of the specific solution found by COBYLA. Crucially, the AMGPRA local analysis converged with a substantially lower computational budget, which is consistent with the theoretical efficiency gains that motivated its development over `mfEGRA`.

8.3 Numerical experiments and results on `solar 10.1`

Having established the probabilistic formulation of the `solar 10.1` reliability problem and the multi-fidelity setup, we now report the numerical results obtained by running AMGPRA across the seven overrun levels $\alpha \in \{0.1\%, 1\%, 5\%, 10\%, 20\%, 30\%, 60\%\}$, with the aim of constructing a complete fragility curve for the CSP plant around its reference optimum.

8.3.1 Experimental setup

For each threshold level $\tau(\alpha) = F(\mathbf{x}_{\text{opt}}) \times (1+\alpha)$, AMGPRA is initialised with a Latin Hypercube sample of 10 points drawn from the Gaussian distribution $\mathcal{N}(\mathbf{x}_{\text{opt}}, \mathbf{\Sigma})$, where $\mathbf{\Sigma} = \text{diag}(\sigma_i^2)$ and $\sigma_i = 0.01 \times x_{\text{opt},i}$. Both fidelity levels ($\phi = 0.5$ and $\phi = 1.0$) are seeded at initialisation, so that the multi-fidelity Kriging model (implemented via `MFK` from the `SMT` toolbox) has a coherent two-level training set from the first iteration.

The active learning loop runs for 15 iterations. At each iteration, the *U-function* acquisition criterion[17]

$$U(\mathbf{x}) \;=\; \frac{|\hat{\mu}(\mathbf{x})|}{\hat{\sigma}(\mathbf{x})}, \tag{38}$$

is minimised over a fixed Monte Carlo population $\mathcal{S}$ of $N = 10{,}000$ points drawn once from the input distribution. The point $\mathbf{x}^* = \arg\min_{\mathbf{x}\in\mathcal{S}} U(\mathbf{x})$ is the most ambiguous candidate, because it is the one for which the surrogate is least certain about the sign of $g(\mathbf{x}) = \tau - F(\mathbf{x})$. The fidelity level assigned to this new evaluation follows a simple variance-based rule: low fidelity ($\phi = 0.5$) is selected when the predictive standard deviation $\hat{\sigma}(\mathbf{x}^*)$ exceeds 0.1, indicating that a cheap exploration call is sufficient; high fidelity ($\phi = 1.0$) is used otherwise, concentrating the expensive budget near the limit-state boundary. The probability of failure is finally estimated by:

$$\hat{P}_f \;=\; \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}_{\hat{\mu}(\mathbf{x}_i)\,\leq\,0}. \tag{39}$$

[17] See **sub-sub-section 4.1.1**.

Acquisition criterion and fidelity allocation: The active learning loop employs the *U-function* [17], defined by Eq. (38), as its acquisition criterion. Although the original AMGPRA formulation [1] uses the EFF, the U-function is a well-established alternative and proves more numerically stable in the highly localized setting of this study, where the 1% Gaussian uncertainty concentrates the candidate population tightly around $\mathbf{x}_{\text{opt}}$. The present implementation is therefore best understood as a *multi-fidelity extension of AK-MCS*, preserving the active learning and two-level Kriging structure of AMGPRA.

The fidelity level assigned to the selected point $\mathbf{x}^*$ follows a simple variance-based rule: a low-fidelity call ($\phi = 0.5$) is used when $\hat{\sigma}(\mathbf{x}^*) > 0.1$, indicating an under-explored region where cheap evaluations suffice to reduce global uncertainty; a high-fidelity call ($\phi = 1.0$) is reserved for well-explored regions where residual uncertainty is localized near the limit-state boundary and requires precise resolution. This division of labor (exploration via LF, exploitation via HF) is the defining feature of multi-fidelity active learning we introduced earlier, and the primary source of computational savings over single-fidelity AK-MCS.

8.3.2 Fragility curve analysis: evolution of P_f with the overrun margin

Table 4 summarises the estimated failure probabilities obtained with the AMGPRA procedure for each threshold level, alongside the corresponding value of τ.

Table 4: Estimated failure probabilities $\hat{P}_f$ as a function of the budget overrun parameter α, for the `solar 10.1` instance with $F(\mathbf{x}_{\text{opt}}) \approx 42.42$, obtained by using the AMGPRA method.

Overrun α (%)	Threshold τ	$\hat{P}_f$	Regime
0.1	42.46	3.95×10^{-2}	Active boundary
1	42.84	7.20×10^{-2}	Active boundary
5	44.54	1.28×10^{-2}	Decreasing
10	46.66	1.00×10^{-3}	Near-zero
20	50.90	≈ 0	Robust zone
30	55.14	≈ 0	Robust zone
60	67.87	≈ 0	Robust zone

The corresponding fragility curve is displayed in Figure 12.

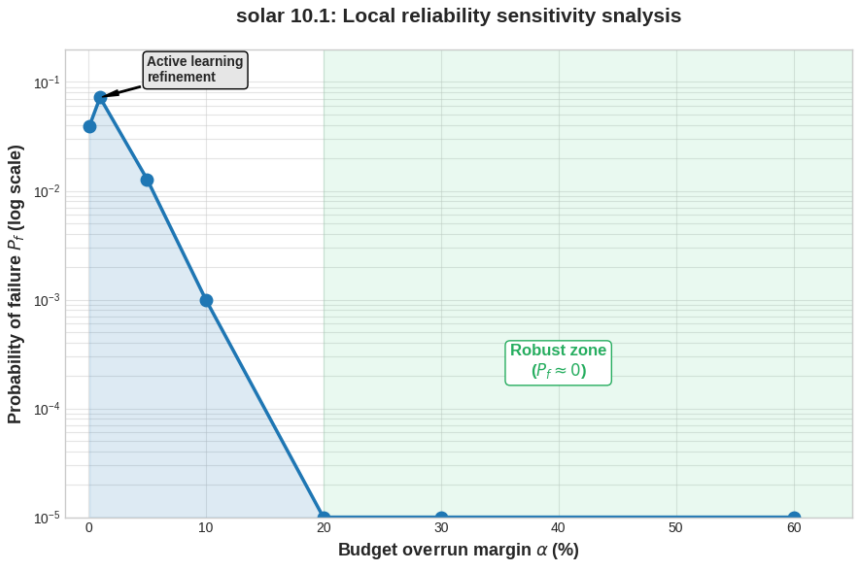

Figure 12: Fragility curve for the `solar 10.1` instance. The vertical axis reports $\hat{P}_f$ on a logarithmic scale. The shaded green region ($\alpha \geq 20\%$) denotes the *robust zone*, where the failure probability is numerically indistinguishable from zero under the 1% Gaussian uncertainty model.

Interpretation of the fragility curve: Several physically meaningful features emerge from Figure 12 and Table 4.

First, the *non-monotone behaviour* between $\alpha = 0.1\%$ and $\alpha = 1\%$ (where $\hat{P}_f$ increases from 3.95×10^{-2} to 7.20×10^{-2} before decreasing) is a signature of an **active learning refinement**. When the threshold is set at the tightest margin ($\alpha = 0.1\%$), the limit-state surface nearly coincides with the mode of the input distribution, which is a particularly delicate configuration for any surrogate-based estimator: the Gaussian process has limited initial information near the optimum, and the first iterations of the U-function criterion are devoted to reducing variance in this region. The surrogate built for $\alpha = 1\%$, having explored a slightly wider boundary, provides a more accurate classification of the 10^4 Monte Carlo samples, hence the corrected (and slightly higher) estimate of $\hat{P}_f$. This phenomenon is indicated by the annotation in Figure 12 and reflects a genuine statistical correction rather than a physical paradox.

Second, the rapid decay of $\hat{P}_f$ between $\alpha = 1\%$ and $\alpha = 10\%$ is consistent with the structure of the `solar` cost function: the penalisation scheme of F creates steep gradients near the feasibility boundary, so that once the threshold is raised a few percent above the optimum, almost all realisations of the 1%-uncertainty distribution fall within the safe domain. This is precisely the regime where AMGPRA is most efficient: the limit-state surface is narrow, the probability mass near the boundary is small, and the active acquisition of low-fidelity points allows a cheap and accurate localisation of that boundary.

Third, for $\alpha \geq 20\%$, the estimated failure probability collapses to zero. This establishes a *robust zone*: under the adopted uncertainty model, a budget tolerance of 20% or more above the optimum cost is sufficient to guarantee that no realisation of the parameter uncertainty triggers a failure event. This result has direct industrial relevance: it suggests that a project manager who sets a cost envelope 20% above the COBYLA optimum faces a negligible risk of overrun under typical manufacturing tolerances of 1%.

8.3.3 2D reliability mapping around the optimum

To complement the scalar fragility curve with a geometric understanding of the failure domain, we project the Monte Carlo population $\mathcal{S}$ onto the two most physically informative dimensions: the receiver outlet temperature $T_{\text{rcv}}^{\text{out}}$ (x_4) and the hot storage tank height H_{hot} (x_1). Although the reliability analysis is conducted in the full five-dimensional input space $\mathcal{X} \subset \mathbb{R}^5$, this planar projection captures the dominant directional sensitivity of F and remains the standard visualisation practice in high-dimensional reliability analysis.

Tight threshold ($\alpha = 1\%$, $\tau = 42.84$): Figure 13 presents the classification of the 10,000 Monte Carlo samples at the +1% threshold. Failure points (red) account for approximately 7.2% of the population, in perfect agreement with the estimated $\hat{P}_f$ reported in Table 4.

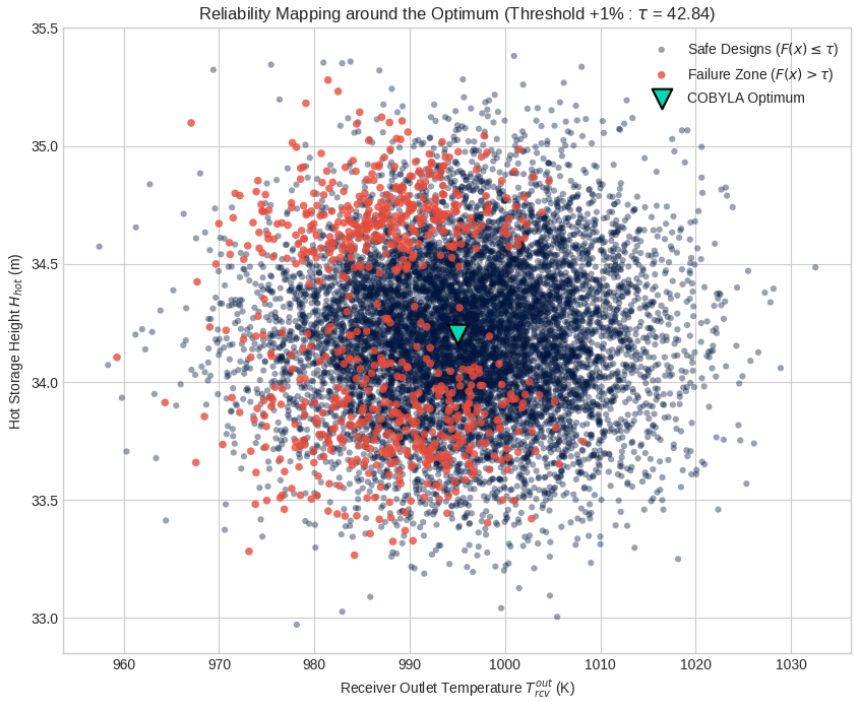

Figure 13: 2D reliability mapping for $\alpha = 1\%$ ($\tau = 42.84$). Each point represents one Monte Carlo sample from $\mathcal{N}(\mathbf{x}_{\text{opt}}, \mathbf{\Sigma})$, classified by the AMGPRA surrogate as safe ($F(\mathbf{x}) \leq \tau$, blue) or failed ($F(\mathbf{x}) > \tau$, red). The cyan triangle marks the COBYLA reference optimum $\mathbf{x}_{\text{opt}}$. Axes display the projection onto ($T_{\text{rcv}}^{\text{out}}$, H_{hot}), while the remaining three variables are present in the surrogate prediction but not displayed.

Two observations are noteworthy. On the one hand, the COBYLA optimum lies at the geometric centre of the distribution, as expected, since the input law is centred on $\mathbf{x}_{\text{opt}}$, and is well within the safe region. This confirms that the reference design is not located near any performance cliff. On the other hand, the failure points are not uniformly distributed around the cloud: they appear preferentially on the left flank (low $T_{\text{rcv}}^{\text{out}}$) and on the lower boundary (low H_{hot}), suggesting that the solar cost function is more sensitive to downward deviations in receiver temperature and storage height than to upward deviations. This asymmetry is a direct consequence of the penalised structure of F: physical constraint violations are more likely to be triggered when the system operates below its nominal thermal capacity.

Large tolerance ($\alpha = 60\%$, $\tau = 67.87$): Figure 14 shows the same projection at the +60% threshold. At this tolerance level, all 10^4 samples are classified as safe, consistently with $\hat{P}_f = 0$ in Table 4. This all-blue scatter plot is not a numerical artefact but a genuine physical result: a 1% uncertainty on the design parameters cannot, in any of the sampled scenarios, push the total cost of the CSP plant to a level 60% above its minimum. The visual contrast between Figures 13 and 14 conveys, perhaps more intuitively than the fragility curve alone, the notion of the *robust zone*: as the overrun tolerance grows, the limit-state surface retreats far from the support of the input distribution, and the failure probability decays to zero.

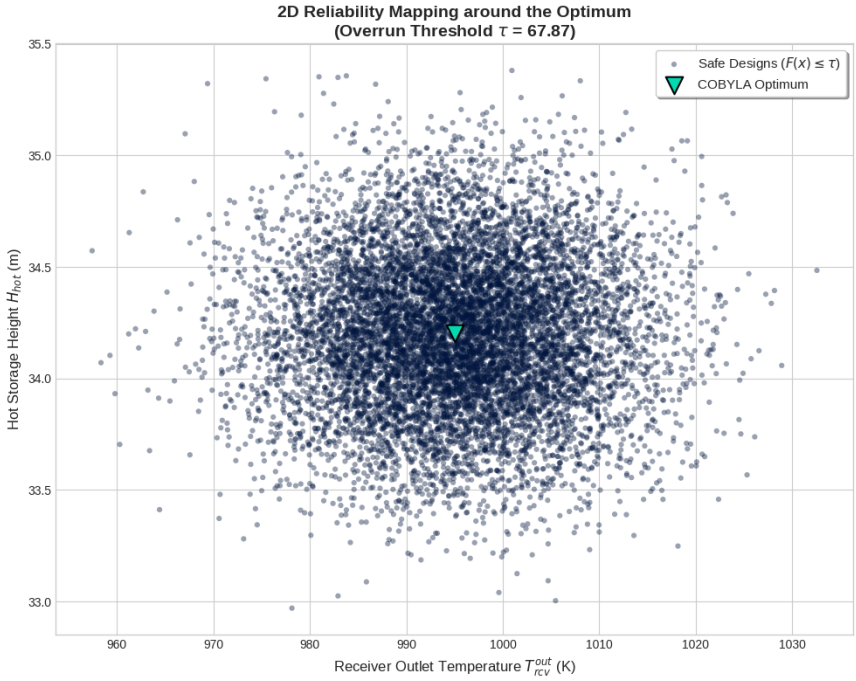

Figure 14: 2D reliability mapping for $\alpha = 60\%$ ($\tau = 67.87$). All Monte Carlo samples lie within the safe domain, consistently with $\hat{P}_f = 0$. This plot illustrates the robust zone identified in the fragility curve: under 1% parametric uncertainty, a 60% cost tolerance is entirely unattainable.

8.3.4 Discussion

The results above allow us to draw several conclusions regarding both the `solar 10.1` problem and the behaviour of the AMGPRA procedure in this setting.

From a reliability analysis point of view, the COBYLA optimum exhibits strong local robustness: the probability of exceeding the nominal cost by more than 10% is of order 10^{-3}, and becomes indistinguishable from zero beyond a 20% margin. This robustness is not trivially obvious from the optimization formulation alone: it is precisely the kind of quantitative insight that a reliability analysis adds beyond a pure minimum-seeking approach.

From a methodological standpoint, the non-monotone behavior of $\hat{P}_f$ at tight thresholds underscores the importance of active learning in this regime. A naive Monte Carlo estimator on this scale ($N = 10^4$) would require $\hat{P}_f \gg 1/N = 10^{-4}$ to produce a reliable estimate. AMGPRA circumvents this limitation by constructing a surrogate that classifies points with high accuracy near the limit-state surface, rather than relying on raw sample counts.

From a multi-fidelity standpoint, the two-level hierarchy ($\phi \in \{0.5, 1.0\}$) proved sufficient to cover the full range of α values within a tractable computational budget. The low-fidelity level contributes the bulk of the exploratory evaluations, while the high-fidelity level is reserved for the final refinement of the boundary, a division of labor that is visible in the convergence behavior of the U-function criterion across iterations.

We emphasize that the present study constitutes a proof-of-concept application of AMGPRA to an industrial-grade simulator. The implementation is transparent about its limitations: the 15-iteration active learning budget is modest, the fidelity allocation rule is heuristic rather than optimised, and the Monte Carlo population is fixed across all threshold levels. These choices were dictated by the computational constraints of our coding environment and by the pedagogical objective of the project. A production-grade deployment would benefit from a larger budget, a formal cost-optimal fidelity allocation policy (as derived in the original AMGPRA paper), and a cross-validation procedure to quantify the uncertainty on $\hat{P}_f$ itself.

9 Concluding remarks

To conclude, surrogate models are widely used in reliability analysis to replace costly numerical simulations by fast-to-evaluate approximations. In particular, multi-fidelity Gaussian process metamodels extend classical GP models by combining a limited number of expensive high-fidelity evaluations with a larger amount of cheaper low-fidelity information. Within this framework, the AMGPRA method was proposed to efficiently guide the active learning process by jointly selecting both the next evaluation point and the appropriate fidelity level, taking into account prediction uncertainty and computational cost. A key contribution of AMGPRA lies in the introduction of the collective learning function (CLF). Unlike standard learning functions that focus on local improvements, CLF aims at quantifying the global impact of adding a new training point. This strategy enables a simultaneous and cost-aware selection of the sampling location and the information source, leading to a more efficient data acquisition process than sequential approaches such as mfEGRA. Numerical results presented in the paper show that AMGPRA can achieve similar or improved accuracy while significantly reducing the overall computational cost.

In this work, we successfully implemented AMGPRA and mfEGRA using the `MFK` model from the `SMT` package. While our implementation differs slightly from the original formulations, it yielded numerical results consistent with the reference benchmark studies, validating our approach. Furthermore, our experiments highlighted a critical practical limitation: the sensitivity of these methods to the relationship between fidelity levels. When low-fidelity models share a clear additive correlation with the high-fidelity response, both algorithms perform efficiently. However, if the low-fidelity contours are shifted or structurally altered, the computational cost increases substantially and the boundary reconstruction degrades. This demonstrates that these multi-fidelity approaches rely heavily on a strong, predictable discrepancy structure to remain cost-effective.

For concrete industrial validation, we applied AMGPRA to the `solar 10.1` benchmark, reformulating the optimization task into a local reliability problem. By evaluating cost overrun probabilities across various thresholds under a one-percent input uncertainty, we generated a fragility curve confirming the optimum's high robustness. The failure probability drops to zero beyond a twenty-percent tolerance margin, and our bi-fidelity setup (combining levels 0.5 and 1.0) successfully covered this range within a strict computational budget. Nevertheless, our current implementation relies on a heuristic fidelity allocation rule rather than a fully optimized one. Future efforts should adopt the formal cost-aware allocation policy from the original AMGPRA framework and propagate metamodel uncertainty to provide confidence bands on the fragility curve, which would considerably strengthen its industrial relevance.

Several perspectives naturally arise from this work. First, the CLF framework could be enriched by incorporating alternative learning functions beyond EFF and the U-function currently used. Criteria based on entropy, stepwise uncertainty reduction, or variance-reduction strategies may provide more informative sampling decisions, especially for complex or highly nonlinear limit-state functions.

Another important limitation concerns the use of Gaussian Process models in high-dimensional settings. As the input dimension increases, GP metamodels may suffer from scalability issues or numerical instability, which reduce predictive performance. Future improvements of AMGPRA could therefore rely on scalable GP variants, or on the integration of dimensionality-reduction techniques. Finally, several parameters in the algorithm, such as stopping criteria threshold, are fixed empirically. The sensitivity of the method to these choices is not fully explored, which could impact its robustness in different applications.

*****

A Additional material on the AMGPRA method

A.1 Closed form expression of the EFF learning function

In the single-fidelity case, the Expected Learning Function (EFF) introduced in the EGRA [18] method admits a closed form expression, which can be derived from Equation (7). For an untried point $\mathbf{x}^* \in \mathcal{X}$, EFF is defined as

$$EFF(\mathbf{x}^*) = \int_{a-\epsilon(\mathbf{x}^*)}^{a+\epsilon(\mathbf{x}^*)} \Big(\epsilon(\mathbf{x}^*) - |a-y|\Big)\, \phi(y\,; \mu_{GP}(\mathbf{x}^*), \sigma_{GP}(\mathbf{x}^*))\, \mathrm{d}y, \tag{40}$$

where $\phi(\cdot)$ denotes the standard normal probability density function.

This integral can be evaluated analytically, leading to the following expression:

$$\begin{aligned} EFF(\mathbf{x}^*) = (\mu_{GP}(\mathbf{x}^*) - a)&\Big[2\Phi\Big(\frac{a-\mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big) - \Phi\Big(\frac{a^- - \mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big) - \Phi\Big(\frac{a^+ - \mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big)\Big] \\ &- \sigma_{GP}(\mathbf{x}^*)\Big[2\phi\Big(\frac{a-\mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big) - \phi\Big(\frac{a^- - \mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big) - \phi\Big(\frac{a^+ - \mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big)\Big] \\ &+ 2\sigma_{GP}(\mathbf{x}^*)\Big[\Phi\Big(\frac{a^+ - \mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big) - \Phi\Big(\frac{a^- - \mu_{GP}(\mathbf{x}^*)}{\sigma_{GP}(\mathbf{x}^*)}\Big)\Big], \end{aligned} \tag{41}$$

where $\Phi(\cdot)$ denotes the standard normal cumulative distribution function, $a = 0$ is the limit-state threshold, $\epsilon(\mathbf{x}^*) = 2\sigma_{GP}(\mathbf{x}^*)$, and $a^{\pm} = a \pm \epsilon(\mathbf{x}^*)$.

The availability of a closed form expression is a major computational advantage, as it avoids numerical integration and enables an efficient evaluation of the learning function over large candidate sets. This closed form expression is used throughout the AMGPRA algorithm to evaluate the EFF-based convergence criterion efficiently, as explained in **sub-section 5.3**.

A.2 Likelihood function optimization and hyperparameters estimation

In the AMGPRA framework, the surrogate model is formulated as a single global multi-fidelity Gaussian process defined on the augmented input space

$$\mathcal{Z} = \{0, \ldots, K\} \times \mathcal{X},$$

where $l = 0$ denotes the high-fidelity model and $l = 1, \ldots, K$ the low-fidelity models.

The construction starts from a GP prior for the high-fidelity response,

$$\hat{g}(0, \mathbf{x}) \sim \mathcal{GP}\Big(\mu_0(\mathbf{x}), \Sigma_0(\mathbf{x}, \mathbf{x}')\Big),$$

and, for each low-fidelity level $l = 1, \ldots, K$, from an independent GP prior for the discrepancy

$$\delta_l(\mathbf{x}) = \hat{g}(l, \mathbf{x}) - \hat{g}(0, \mathbf{x}) \sim \mathcal{GP}\Big(\mu_l(\mathbf{x}), \Sigma_l(\mathbf{x}, \mathbf{x}')\Big).$$

Following the assumptions adopted in the article, we take $\mu_l(\mathbf{x}) = 0$ for all $l \geq 1$, so that

$$\hat{g}(l, \mathbf{x}) = \hat{g}(0, \mathbf{x}) + \delta_l(\mathbf{x}), \qquad l = 1, \ldots, K.$$

As a result, for any finite collection of source-location pairs

$$\mathbf{z}^{(i)} = (l_i, \mathbf{x}^{(i)}) \in \mathcal{Z}, \qquad i = 1, \ldots, N,$$

the stacked observation vector

$$\mathbf{y} = \Big(y_1, \ldots, y_N\Big)^\top, \qquad \text{with} \qquad y_i = g_{l_i}\Big(\mathbf{x}^{(i)}\Big),$$

follows a multivariate normal distribution:

$$\mathbf{y} \sim \mathcal{N}_N\Big(\mathbf{m}_\theta, \mathbf{K}_\theta\Big), \tag{42}$$

where the mean vector is defined by

$$[\mathbf{m}_\theta]_i = \mu_{\mathrm{pr}}\Big(\mathbf{z}^{(i)}\Big) = \mu_0\Big(\mathbf{x}^{(i)}\Big),$$

since the discrepancy means are taken equal to zero, and where the covariance matrix is given by

$$[\mathbf{K}_\theta]_{i,j} = \Sigma_{\mathrm{pr}}\Big(\mathbf{z}^{(i)}, \mathbf{z}^{(j)}\Big).$$

Under the independence assumptions between the high-fidelity GP and the discrepancy processes, the joint prior covariance writes

$$\Sigma_{\mathrm{pr}}\Big((l, \mathbf{x}), (l', \mathbf{x}')\Big) = \Sigma_0(\mathbf{x}, \mathbf{x}') + \mathbf{1}_{\{l=l'>0\}}\, \Sigma_l(\mathbf{x}, \mathbf{x}'), \tag{43}$$

where $\mathbf{1}_{\{l=l'>0\}}$ denotes the indicator function equal to 1 when $l = l' > 0$ and 0 otherwise. Hence, all observations from all fidelity levels are coupled through the same global covariance structure.

The likelihood function is therefore the joint density of the complete observation vector $\mathbf{y}$, viewed as a function of the hyperparameters θ. The associated log-likelihood is

$$\log \mathcal{L}(\theta) = -\frac{1}{2}\Big(\mathbf{y} - \mathbf{m}_\theta\Big)^\top \mathbf{K}_\theta^{-1}\Big(\mathbf{y} - \mathbf{m}_\theta\Big) - \frac{1}{2}\log\Big|\mathbf{K}_\theta\Big| - \frac{N}{2}\log(2\pi). \tag{44}$$

In the isotropic setting considered here, each covariance kernel is parameterized by a variance parameter and a single correlation length. Accordingly, the full hyperparameter vector can be written as

$$\theta = \{\mu_0, \sigma_0, h_0, \sigma_1, h_1, \ldots, \sigma_K, h_K\},$$

where (σ_0, h_0) are associated with the high-fidelity kernel Σ_0, and (σ_l, h_l) with the discrepancy kernel Σ_l for $l = 1, \ldots, K$.

The hyperparameters are estimated by maximum likelihood, that is, by solving

$$\hat{\theta} \in \arg\max_\theta \log \mathcal{L}(\theta).$$

As in standard GP regression, this optimization does not admit a closed-form solution because it involves the inverse and determinant of the joint covariance matrix $\mathbf{K}_\theta$. Therefore, numerical optimization procedures are used in practice, typically under bound constraints to ensure numerical stability.

A.3 On the implementation of the AK-MCS method

The implementation of the AK-MCS algorithm is relatively simple, as it relies on a single level of fidelity corresponding to the high-fidelity limit-state function $g : \mathcal{X} \to \mathbb{R}$. Starting from an initial DoE $\mathcal{D}_n = \{(x_i, g(x_i))\}_{i=1}^n$, a Gaussian process surrogate model is trained. At each iteration, the posterior mean $\hat{\mu}_{GP}(x)$ and variance $\hat{\sigma}^2_{GP}(x)$ are evaluated over a Monte Carlo population $\mathcal{S} \subset \mathcal{X}$.

The EFF, denoted $\eta(x)$, is then computed for all $x \in \mathcal{S}$ in order to identify the most informative point:

$$x^\star = \arg\max_{x \in \mathcal{S}} \eta(x).$$

This point is evaluated using the true function g, added to the DoE, and the surrogate model is updated. The procedure is repeated until a convergence criterion is satisfied, typically based on the maximum value of the learning function.

Since only the high-fidelity model is considered, the algorithm does not involve any selection between multiple sources of information, which significantly simplifies both the modeling and enrichment steps compared to multi-fidelity approaches such as AMGPRA.

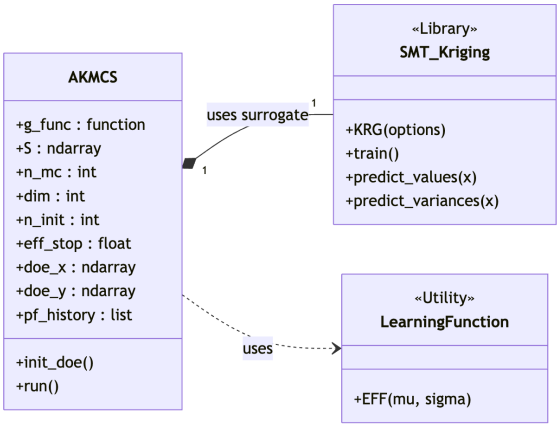

Figure 15: UML class diagram of the AKMCS implementation

References

[1] C. Zhang, C. Song, and A. Shafieezadeh, *Adaptive Reliability Analysis for Multi-fidelity Models using a Collective Learning Strategy*, in "Structural Safety", vol.94, 2022.

[2] A. Chaudhuri, A. N. Marques, and K. E. Willcox, *mfEGRA: Multifidelity Efficient Global Reliability Analysis*, 2020.

[3] O. Ditlevsen, and HO. Madsen, *Structural reliability methods*, John Wiley & Sons, 1996.

[4] V. J. Romero, L. P. wiler, and A. A. Giunta, *Construction of response surfaces based on progressive-lattice sampling experimental designs with application to uncertainty propagation*, in "Structural Safety", vol.26, no.2, 2004.

[5] J.-M. Bourinet, *Rare-event probability estimation with adaptive support vector regression surrogates*, in "Reliability Engineering and System Safety", vol.150, pp.210–221, 2016.

[6] M. Kennedy and A. O'Hagan, *Predicting the Output from a Complex Computer Code When Fast Approximations Are Available*, in "Biometrika", vol.87, no.1, pp.1–13, 2000.

[7] L. L. Gratiet, *Bayesian analysis of hierarchical multi-fidelity codes*, ArXiv11125389 Math Stat, Sep. 2012, Accessed: Sep. 30, 2020. [Online]. Available: http://arxiv.org/abs/1112.5389

[8] R. Y. Rubinstein and D. P. Kroese, *Simulation and the Monte Carlo method*, John Wiley & Sons, 2016.

[9] A.I.J. Forrester, A. Sóbester, and A.J. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*, John Wiley & Sons, 2008.

[10] C.E. Rasmussen, and C.K.I. Williams, *Gaussian Processes for Machine Learning*, 2005.

[11] N.V. Queipo, R.T Haftka, W. Shyy, T. Goel, R. Vaidyanathan, and P.K Tucker, *Surrogate-based analysis and optimization*, in "Progress in Aerospace Sciences", vol.41, no.1, pp.1–28, 2005.

[12] Michael L. Stein, *Interpolation of Spatial Data: Some Theory for Kriging*, Springer, 1999.

[13] P.Z.G. Qian and C.F.J. Wu, *Bayesian Hierarchical Modeling for Integrating Low-accuracy and High-accuracy Experiments*, in "Technometrics", vol.50, no.2, 2008.

[14] Donald R. Jones, M. Schonlau, and W.J Welch, *Efficient Global Optimization of Expensive Black-Box Functions*, in "Journal of Global Optimization", vol.13, pp.455-492, 1998.

[15] Z. Wang, L. Zhang, and B. Sudret, *Active learning Kriging-based reliability analysis: A review*, in "Structural Safety", vol.76, pp.55–69, 2019.

[16] C. G. Bucher and U. Bourgund, *A fast and efficient response surface approach for structural reliability problems*, in "Structural Safety", vol. 7, no. 1, pp. 57–66, 2009.

[17] B. Echard, N. Gayton, and M. Lemaire, *AK-MCS: An active learning reliability method combining Kriging and Monte Carlo Simulation*, in "Structural Safety", vol.33, no.2, 2011.

[18] B. J. Bichon, M. . Eldred, L. P. wiler, . Mahadevan, and J. M. McFarland, *Efficient global reliability analysis for nonlinear implicit performance functions*, in "AIAA J.", vol. 46, no. 10, 2008.

[19] P. Perdikaris, M. Raissi, A. Damianou, N. D. Lawrence, G. E. Karniadakis, *Nonlinear information fusion algorithms for data-efficient multi-fidelity modelling*, The Royal Society, 2017.

[20] M. Poloczek, J. Wang, and P. Frazier, *Multi-Information Source Optimization*, in "Advances in Neural Information Processing Systems 30", 2017.

[21] C. Devon Lin and B. Tang, *Latin Hypercubes and Space-filling Designs*, ArXiv2203.06334v1 Stat ME, Mar. 2022, Accessed: Nov. 30 2025.[Online] Available: https://arxiv.org/abs/2203.06334v1

[22] M. A. Bouhlel, *Optimisation auto-adaptative en environnement d'analyse multidisciplinaire via les modèles de krigeage combinés à la méthode PLS*, nstitut Supérieur de l'Aéronautique et de l'Espace, 2016.

[23] N. Andrés-Thio, C. Audet, M. Diago, A. E. Gheribi, S. Le Digabel, X. Lebeuf, M. Lemyre Garneau, C. Tribes, *solar: A solar thermal power plant simulator for blackbox optimization benchmarking*, ArXiv:2406.00140v1, May. 2024, Accessed: Feb. 20 2026.[Online] Available: https://arxiv.org/pdf/2406.00140

[24] M.J.D. Powell, *A direct search optimization method that models the objective and constraint functions by linear interpolation*, Advances in Optimization and Numerical Analysis, 1994.